

UNIVERSITATEA NAȚIONALĂ DE ȘTIINȚĂ ȘI TEHNOLOGIE
POLITEHNICA DIN BUCUREȘTI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL DE CALCULATOARE



PROIECT DE DIPLOMĂ

CultureQuest - Platforma mobilă de explorare urbană bazată pe proximitate, cu recomandări personalizate prin învățare federată

Andrei Cozma

Coordonator științific:

Prof. dr. ing. Ciprian-Mihai Dobre
Prof. dr. ing. Radu-Ioan Ciobanu

BUCUREȘTI

2026

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY
POLITEHNICA BUCHAREST
FACULTY OF AUTOMATIC CONTROL AND COMPUTERS
COMPUTER SCIENCE AND ENGINEERING DEPARTMENT



DIPLOMA PROJECT

CultureQuest - Proximity-Based Mobile Platform for Urban
Exploration with Personalized Recommendations via Federated
Learning

Andrei Cozma

Thesis advisor:

Prof. dr. ing. Ciprian-Mihai Dobre
Prof. dr. ing. Radu-Ioan Ciobanu

BUCHAREST

2026

CUPRINS

1	Introducere	1
1.1	Context	1
1.2	Definirea problemei	1
1.3	Obiective	2
1.4	Soluția propusă	2
1.5	Rezultatele obținute	3
1.6	Structura lucrării	3
2	Analiza Cerințelor	5
2.1	Utilizatori țintă și cazuri de utilizare	5
2.2	Cerințe funcționale	5
2.3	Cerințe non-funcționale	6
2.4	Domeniul de aplicare al proiectului	7
3	Soluții Existente	8
3.1	Aplicații existente pentru explorare urbană	8
3.2	Analiză comparativă între abordările centralizate și învățarea federată pentru recomandări	10
3.3	Învățare federată: Stadiul actual	10
3.4	Alternative respinse	11
3.5	Tehnologiile utilizate și justificarea acestora	12
3.5.1	Flutter și Riverpod	13
3.5.2	FastAPI, MongoDB și Redis	13
3.5.3	Motorul de rutare OSRM	14
3.5.4	Perceptron multi-strat	14
4	Soluția Propusă	16
4.1	Prezentare generală a sistemului	16

4.2	Arhitectura aplicației mobile	17
4.3	Arhitectura serviciilor <i>backend</i>	17
4.4	Proiectarea sistemului de învățare federată	18
4.4.1	Arhitectura modelului	18
4.4.2	Antrenarea locală	20
4.4.3	Agregarea asincronă și reducerea în funcție de vechime	20
4.4.4	Limitarea gradientilor	21
4.4.5	Confidențialitate diferențială	22
4.4.6	Controlul concurenței	22
4.5	Fluxul de generare a rutelor	23
4.6	Proiectarea elementelor de gamificare	24
4.7	Harta și navigarea	25
4.8	Arhitectura de confidențialitate	26
5	Detalii de Implementare	28
5.1	Implementarea aplicației mobile	28
5.1.1	Serviciul client de învățare federată	28
5.1.2	Adăugarea de zgomot pentru asigurarea confidențialității diferențiale	29
5.1.3	Bufferul local de interacțiuni și persistența datelor	29
5.1.4	Stocarea temporară a obiectivelor pentru funcționarea offline	30
5.1.5	Algoritmul de generare a rutelor	30
5.1.6	Sistemul de misiuni și tehnologia de <i>geofencing</i>	31
5.2	Implementarea serviciilor <i>backend</i>	32
5.2.1	Serviciul de agregare	32
5.2.2	Punctele de acces API	32
5.2.3	Sistemul de recenzii și moderarea conținutului	32
5.3	Fluxul etapei de pre-antrenare	32
5.4	<i>Framework-ul</i> pentru simularea procesului de învățare federată	32

5.5	Dificultăți întâmpinate în procesul de implementare	32
6	Evaluare	33
6.1	Corectitudinea funcțională	33
6.2	Performanța modelului de învățare federată	33
6.2.1	Rezultatele etapei de pre-antrenare	33
6.2.2	Rezultatele simulării procesului de învățare federată	33
6.3	Evaluarea confidențialității, robusteții și scalabilității	33
6.4	Analiză comparativă și discuții	33
7	Concluzii	34
7.1	Concluzii generale	34
7.2	Direcții de cercetare și lucrări viitoare	34
	Bibliografie	35
	Anexe	37
	Anexa A Extrase de Cod	40

SINOPSIS

CultureQuest este o platformă mobilă de explorare urbană bazată pe proximitate, oferind recomandări personalizate legate de obiectivele turistice și recreative dintr-un oraș, fără ca datele utilizatorului să părăsească dispozitivul. Problema pe care o tratez este că aplicațiile de tip *city-guide* fie oferă aceleași sugestii tuturor, indiferent de interese, fie personalizează recomandările prin salvarea centralizată a istoricului de interacțiuni cu obiectivele sau monitorizarea traseului, ceea ce ridică probleme din punctul de vedere al confidențialității. Pentru a evita acest compromis, am implementat o aplicație Flutter în care datele sunt salvate doar local, interacțiunile vizibile altor utilizatori precum recenziile sunt anonime și adăugate strict de vizitatori, iar rutele turistice generate sunt individualizate prin intermediul unei rețele neurale de tip perceptron multi-strat (MLP, *Multi-Layer Perceptron*). Modelul global inițial este pre-antrenat pe seturi de date publice, iar acesta este modificat și îmbunătățit prin tehnici de învățare federată (FL, *Federated Learning*) rulate asincron pe dispozitivele clienților. Recomandările de rute combină scorul oferit de model pentru o atracție turistică cu proximitatea acesteia, iar un sistem de misiuni gamificate încurajează explorarea și colectarea de noi date. Printr-o simulare care include mai multe runde - mai exact 600 - se confirmă funcționarea fluxului FL în mod corespunzător.

ABSTRACT

CultureQuest is a proximity-based mobile urban exploration platform, offering personalized recommendations for tourist and recreational attractions in a city, without the user's data leaving the device. The problem I address is that city-guide type applications either offer the same suggestions to all users, regardless of interests, or personalize recommendations by centrally storing a user's interaction history with landmarks or by tracking their route, both of which raise privacy concerns. To avoid this trade-off, I developed a Flutter application where data is saved only locally, interactions visible to other users such as reviews are anonymous and can only be added by actual visitors, and the generated tourist routes are personalized through a multi-layer perceptron (MLP) neural network. The initial global model is pre-trained on public datasets, and it is modified and improved through federated learning (FL) techniques run asynchronously on the client devices. Route recommendations combine the model's score for a tourist attraction with its proximity, while a gamified quest system encourages exploration and data collection. A simulation that includes multiple client rounds - 600, to be exact - confirms that the FL flow is working properly.

1 INTRODUCERE

Acest capitol prezintă contextul în care se situează proiectul și problema care îl motivează, obiectivele propuse, o privire de ansamblu asupra soluției dezvoltate și a rezultatelor obținute, precum și modul în care este organizat restul lucrării.

1.1 Context

Telefoanele mobile au devenit, în ultimii ani, principalul instrument prin care utilizatorii interacționează cu mediul din jurul lor, iar aplicațiile mobile bazate pe locație - de la Google Maps, TripAdvisor și GetYourGuide, la diverse aplicații de tip *city-guide* - au devenit una dintre principalele modalități prin care turiștii și localnicii descoperă obiective turistice și culturale dintr-un oraș și primesc recomandări din ce în ce mai adaptate propriilor interese [6], tendință susținută de creșterea constantă a pieței, cu peste 850 de milioane de utilizatori de aplicații de călătorit la nivel mondial în 2023¹.

Pentru a oferi recomandări personalizate, aceste aplicații se bazează pe colectarea și procesarea centralizată a istoricului de interacțiuni - locurile vizitate și în ce ordine, recenziile adăugate, timpul petrecut în diverse locații, ce categorii culturale îl atrag - pentru a înțelege comportamentul concret al utilizatorului. Astfel, accesul la acest ansamblu de date, coroborat cu accesul la poziția GPS în orice moment, necesară pentru *geofencing* și generarea de trasee, înseamnă că un server centralizat ar acumula un profil detaliat al fiecărui turist, care ar putea dezvălui rutine zilnice, preferințe personale sau chiar afilierea religioasă și orientările politice [7].

Învățarea federată (FL, *Federated Learning*) apare ca alternativă la acest model centralizat: modelul este antrenat local, pe dispozitivul utilizatorului, folosind datele acestuia, iar către server sunt trimise doar actualizările rezultate, nu datele brute, pentru a fi agregate într-un model global. FL este deja folosită la scară destul de mare în sisteme de producție [4, 19, 9], ceea ce motivează aplicarea ei și în cadrul acestei platforme, pentru a oferi recomandări de rute turistice fără a compromite confidențialitatea clienților.

1.2 Definirea problemei

Majoritatea aplicațiilor de explorare urbană dezvoltate oferă, în absența unui profil construit din acțiunile turiștilor pe platformă, aceleași rute tuturor, indiferent de interesele fiecăruia, chiar dacă acestea sunt legate de artă, gastronomie, natură sau alte evenimente culturale.

¹Business of Apps, *Travel App Report 2025*. <https://www.businessofapps.com/data/travel-app-report/> (accesat 19.06.2026).

Această abordare generică este mai simplă de dezvoltat și de întreținut, însă, nu ține cont de faptul că două persoane care vizitează același oraș pot avea preferințe complet diferite în ceea ce privește modul în care doresc să îl exploreze.

Varianta prin care o aplicație devine personalizată presupune, de regulă, ca istoricul de interacțiuni al utilizatorului să fie încărcat și procesat pe un server central, exact tipul de practică care aduce în discuție problemele de confidențialitate menționate mai sus. Utilizatorului i se prezintă, astfel, un compromis: fie acceptă recomandări generice, fie acceptă ca datele sale comportamentale să fie colectate central pentru a primi recomandări adaptate intereselor sale - iar acest compromis este punctul de plecare al proiectului de față.

1.3 Obiective

Pornind de la acest compromis, proiectul de față își propune să arate că personalizarea și păstrarea confidențialității nu se exclud reciproc, prin trei obiective principale:

- (a) O aplicație mobilă prin care utilizatorul poate descoperi obiective culturale dintr-un oraș printr-o explorare bazată pe proximitate, cu un sistem de misiuni (*quest-uri*) și puncte obținute pentru interacțiunile pe platformă, care îl încurajează să viziteze locuri noi.
- (b) Personalizarea recomandărilor de rute pentru fiecare utilizator, fără ca datele acestuia - istoricul de interacțiuni, locațiile vizitate - să părăsească dispozitivul.
- (c) Implementarea unui flux complet de FL: pre-antrenarea modelului global pe date publice și ajustarea fină (*fine-tuning*); antrenarea locală prin coborâre pe gradient stocastică (SGD, *Stochastic Gradient Descent*) conform FedAvg [13], trimiterea spre server doar a variației ponderilor față de modelul global, agregarea asincronă a actualizărilor prin FedAsync [17] și împiedicarea deducerii din acestea a interacțiunilor concrete, prin adăugarea de zgomot Gaussian asupra variației ponderilor.

1.4 Soluția propusă

Soluția dezvoltată este o aplicație mobilă realizată în Flutter, prin care utilizatorul poate vedea pe hartă obiectivele turistice și evenimentele din apropiere. Pe lângă explorare, aplicația oferă un sistem de *quest-uri*, care încurajează vizitarea unor obiective noi, dar și posibilitatea de a lăsa recenzii, povești sau chiar alte *quest-uri* pentru locurile vizitate - tot ce adaugă utilizatorii devine, în timp, parte din imaginea pe care comunitatea o are despre fiecare atracție.

Pentru personalizare, fiecare utilizator are pe dispozitiv un model mic, un perceptron multi-strat (MLP, *Multi-Layer Perceptron*), care reprezintă un scor de recomandare pentru fiecare obiectiv. Pe baza acestui scor, fiecărui obiectiv i se asociază o etichetă care arată cât de recomandat este pentru utilizator, în funcție de interesele acestuia, de ora curentă, de tipul

locului etc., iar la generarea rutei, scorul fiecărui obiectiv este combinat cu distanța față de poziția curentă. Modelul global inițial este pre-antrenat cu ajutorul unor seturi de date din mediul online, apoi antrenat pe dispozitiv prin FL, iar *backend-ul* coordonează învățarea asincronă.

1.5 Rezultatele obținute

La nivel general, rezultatul este o aplicație Flutter funcțională, care reunește harta cu obiective și evenimente din apropiere, generarea de rute personalizate și sistemul de *quest-uri*. Personalizarea descrisă în secțiunea anterioară este vizibilă direct în aplicație, pentru fiecare obiectiv din apropiere și în cadrul rutelor generate.

Pe partea de FL, modelul de pe dispozitiv pornește de la o variantă a modelului global trecută și prin etapa de *fine-tuning*, iar antrenarea locală se face prin FedAvg. Agregarea pe server este asincronă, cu o reducere în funcție de vechimea (*staleness discount*) actualizărilor venite de la clienți [17] - o metodă prin care influența lor asupra modelului global este diminuată. O simulare de 600 de runde cu 10.000 de clienți sintetici arată o reducere a pierderii BCE (*Binary Cross-Entropy*) cu 30,9% (de la 0,695 la 0,480), față de 27,2% în varianta fără discount. Chiar dacă diferența procentuală nu pare foarte semnificativă, simularea a inclus doar 20% clienți cu *staleness-ul* mai mare decât 2, deci raportarea se face la acest procent. Am confirmat, astfel, că ignorarea vechimii actualizărilor contribuie la degradarea convergenței. În plus, eficiența rulării pe dispozitiv este validată de latența de inferență a modelului: ≈ 40 μ s pentru fiecare predicție în simularea NumPy, fără dependențe native adiționale, ceea ce permite personalizarea continuă fără impact perceptibil asupra experienței.

1.6 Structura lucrării

Capitolul 1 a introdus, la nivel general, contextul, problema, obiectivele, soluția propusă și rezultatele obținute. Capitolele următoare intră în detaliu pe fiecare dintre aceste aspecte.

Capitolul 2 stabilește cerințele aplicației, pornind de la cazurile de utilizare ale unui turist sau localnic care explorează un oraș, și le împarte în cerințe funcționale și non-funcționale. Tot aici se delimitează și domeniul de aplicare al proiectului.

Capitolul 3 trece în revistă aplicațiile de explorare urbană existente și modul în care acestea personalizează recomandările, comparând abordările centralizate cu FL. Se poziționează CultureQuest pe baza taxonomiei FL, se explică de ce alte alternative de algoritmi FL nu se potrivesc design-ului asincron al platformei și se justifică tehnologiile alese.

Capitolul 4 detaliază arhitectura propusă a sistemului. Aici se discută structura aplicației mobile și a serviciilor *backend*, proiectarea modelului MLP și a procesului FL, fluxul de generare

a rutelor, elementele de gamificare și arhitectura de confidențialitate.

După proiectare, **Capitolul 5** trece la implementarea efectivă - serviciul client FL și mecanismul de confidențialitate diferențială (DP, *Differential Privacy*), serviciile *backend* de agregare și de moderare a conținutului, etapa de pre-antrenare și *framework-ul* folosit pentru simularea procesului FL.

Capitolul 6 evaluează rezultatele obținute - corectitudinea funcțională a aplicației, performanța modelului rezultată din pre-antrenare și din fluxul FL, efectul DP și al limitării asupra robusteții - și include o discuție comparativă față de alternativele respinse.

Capitolul 7 încheie lucrarea cu o privire de ansamblu asupra obiectivelor stabilite inițial, în raport cu ce a fost dezvoltat și propune direcții viitoare - de exemplu, personalizarea suplimentară per utilizator sau alte tipuri de agregare în ceea ce privește modelul rețelei neurale.

2 ANALIZA CERINȚELOR

Capitolul de față detaliază cerințele care reies din obiectivele stabilite anterior, și anume cine sunt utilizatorii și care sunt cazurile de utilizare care trebuie acoperite, ce funcționalități rezultă de aici, ce constrângeri non-funcționale se fac necesare și ce rămâne în afara domeniului de aplicare. Toate aceste cerințe stau la baza deciziilor de proiectare din Capitolul 4, fiind verificate ulterior în evaluarea din Capitolul 6.

2.1 Utilizatori țintă și cazuri de utilizare

Utilizatorul principal al aplicației este un turist sau un localnic care explorează un oraș - sau o zonă a acestuia - cu dorința de a vizita obiective turistice sau culturale din proximitate, adaptate intereselor proprii. Am identificat patru scenarii care acoperă principalele categorii de utilizatori: turistul aflat pentru prima oară într-un loc necunoscut, cu dorința de a se orienta ușor și de a descoperi obiective potrivite pentru el; localnicul interesat de evenimente și obiective culturale din apropiere, cu dorința de a descoperi conținut nou și de a contribui la comunitate; utilizatorul interesat de confidențialitate, vrând să beneficieze de recomandări personalizate fără ca datele sale să ajungă într-un loc centralizat; administratorul, cu rolul de a valida propunerile comunității și de a menține calitatea informațiilor pentru ceilalți utilizatori. Cerințele au fost validate prin corelarea fiecărei funcționalități cu unul sau mai multe dintre aceste scenarii.

Primul caz de utilizare este cel de generare de rute și suport pe parcursul urmăririi acestora, în care utilizatorul vede pe hartă poziția sa curentă, locurile de interes din apropiere și primește o rută care combină preferințele sale cu acestea. Mai mult, interacțiunile cu aplicația - vizite, *quest-uri* finalizate, recenzii - determină ajustarea, prin FL, a modelului de pe dispozitiv - și, implicit, și a celui global - fără ca istoricul brut al interacțiunilor să ajungă pe server. Un al doilea caz de utilizare este cel de contribuție de conținut, în care utilizatorul poate propune obiective noi, povești, *quest-uri* sau recenzii, toate acestea necesitând aprobarea de către alte persoane care folosesc aplicația sau de către un administrator. Prin conturile de tip administrator, relevante mai ales pentru 2.2, se aprobă sau se respinge conținutul propus de utilizatorii obișnuiți.

2.2 Cerințe funcționale

Pornind de la cazurile de utilizare din secțiunea anterioară, cerințele funcționale au fost sintetizate în **tabelul 1**, fiecare cu o descriere, condiții de acceptare și secțiunile din Capitolele 4-6 unde cerința este tratată. Primul grup (de la **FR-1** la **FR-4**) acoperă bucla principală de

explorare: afișarea pe hartă a obiectivelor turistice din proximitate, generarea rutei care combină scorul modelului de tip MLP cu distanța față de poziția curentă, *quest-urile* disponibile la obiectivele vizitate și posibilitatea de a adăuga sau vizualiza recenzii.

Al doilea grup (de la **FR-5** la **FR-7**) ține de personalizare și administrare: **FR-5** descrie fluxul procesului FL: primirea parametrilor modelului actualizat de la server, urmată de antrenarea locală, fără ca istoricul interacțiunilor să ajungă pe server, flux verificat prin simularea de 600 de runde descrisă în 6.2; **FR-6** descrie controalele de confidențialitate - asigurarea DP, vizibilitatea asupra datelor - prin adăugarea de zgomot Gaussian și limitare a gradientilor aplicate pe actualizările de ponderi, verificat prin analiza compromisului dintre confidențialitate și utilitate din 6.3; iar **FR-7** descrie rolul de administrator - aprobarea sau respingerea conținutului propus, verificat prin demo funcțional în 6.1.

Tabelul 1: Cerințe funcționale cu condiții de acceptare.

ID	Descriere	Condiții de acceptare	Secțiune
FR-1	Obiective din proximitate pe hartă.	<i>I</i> : coordonate GPS. <i>E</i> : markere pentru obiective aflate la sub 200 m. <i>Î</i> : toate obiectivele din raza setată apar pe hartă. <i>T</i> : demo funcțional.	4.7, 5.1.4, 6.1
FR-2	Rute personalizate: scor MLP și proximitate.	<i>I</i> : interese + poziție curentă. <i>E</i> : rută ordonată după scor MLP ponderat cu distanța. <i>Î</i> : rute diferite pentru utilizatori diferiți. <i>T</i> : demo cu utilizatori diferiți.	4.5, 5.1.5, 6.1
FR-3	<i>Quest-uri</i> la obiective vizitate; puncte și progres.	<i>I</i> : intrare în <i>geofence-ul</i> unui obiectiv. <i>E</i> : <i>quest</i> disponibil; puncte la finalizare. <i>Î</i> : <i>quest</i> activat la intrare; puncte actualizate la final. <i>T</i> : demo funcțional.	4.6, 5.1.6, 6.1
FR-4	Adăugare și vizualizare recenzii.	<i>I</i> : text + rating de la utilizator autentificat. <i>E</i> : recenzie vizibilă în cadrul obiectivului. <i>Î</i> : recenzia apare pentru toți utilizatorii. <i>T</i> : demo funcțional.	5.2.3, 6.1
FR-5	Sincronizare model și antrenare locală, fără date pe server.	<i>I</i> : interacțiuni locale. <i>E</i> : ponderi actualizate local; doar actualizarea trimisă la server. <i>Î</i> : pierderea BCE scade cu $\geq 25\%$ față de varianta inițială. <i>T</i> : simulare FL, 600 runde, 10.000 clienți.	4.4, 5.1.1, 5.1.2, 5.1.3, 5.2.1, 6.2
FR-6	Zgomot DP pe actualizări; vizibilitate asupra datelor trimise.	<i>I</i> : actualizare locală a ponderilor. <i>E</i> : actualizare cu zgomot Gaussian și limitare a normei. <i>Î</i> : interacțiunile utilizatorului nu sunt transmise direct. <i>T</i> : compromis confidențialitate, utilitate.	4.8, 5.1.2, 6.3
FR-7	Administrator: aprobă sau respinge conținut.	<i>I</i> : propunere de conținut. <i>E</i> : publicat sau respins de administrator. <i>Î</i> : conținut neaprobat absent din aplicație. <i>T</i> : demo flux administrator.	5.2.3, 6.1

I = Intrare; *E* = leșire; *Î* = Îndeplinit; *T* = Testat în Cap. 6.

2.3 Cerințe non-funcționale

La fel ca în secțiunea 2.2, cerințele non-funcționale au fost sintetizate în **tabelul 2**, cu aceeași structură (descriere și secțiunile din Capitolele 4-6 unde sunt tratate), fiecare însoțită de o metrică verificabilă. Primul grup (de la **NFR-1** la **NFR-3**) are legătură cu constrângerile impuse pe dispozitiv: **NFR-1** cere ca nicio informație privată despre utilizator - exceptând lista de interese - să nu părăsească telefonul, **NFR-2** cere ca obiectivele turistice să rămână disponibile și fără conexiune la internet, iar **NFR-3** cere ca antrenarea locală să nu fie complexă din punct de vedere computațional - ceea ce a motivat alegerea unui model MLP de dimensiuni

reduse, antrenat în 5 epoci locale.

Al doilea grup ține de partea de *backend*: **NFR-4** impune ca serviciul de agregare să suporte actualizări asincrone primite de la mai mulți clienți, fără conflicte între runde, asigurat printr-un *lock* Redis, iar **NFR-5** impune ca utilizatorii să primească recomandări utile încă de la început printr-un model global pre-antrenat [4], și din caracteristici bazate pe ariile de interes asociate fiecărui obiectiv, până când procesul FL își face efectul.

Tabelul 2: Cerințe non-funcționale.

ID	Descriere	Secțiune
NFR-1	Confidențialitate: nicio interacțiune a utilizatorului nu părăsește dispozitivul; doar actualizarea de ponderi (≈ 5 KB la fiecare rundă FL) este trimisă la server.	4.8, 5.1.2, 5.1.3, 6.3
NFR-2	Reziliență offline: obiective turistice stocate în <i>cache</i> local; accesibile fără conexiune la rețea.	5.1.4, 6.1
NFR-3	Cost redus de antrenare pe dispozitiv: model de 1.281 parametri (≈ 5 KB valori în format float32); 5 epoci locale la fiecare rundă FL; calculul scorului pentru un obiectiv (un singur <i>forward pass</i>) durează $\approx 39 \mu s$ (simulare pe laptop).	4.4.1, 5.1.1, 6.2.1
NFR-4	Scalabilitatea <i>backend-ului</i> : un <i>lock</i> Redis serializează actualizările FL asincrone fără conflicte de date; 0 erori la cel mult 10 cereri simultane (latență medie 1,4 s la 10 cereri); degradare la 20 cereri.	4.4.6, 5.2.1, 6.3
NFR-5	<i>Cold start</i> : recomandări utile pentru utilizatori folosind modelul pre-antrenat, fără runde FL prealabile; FL îmbunătățește modelul (pierdere BCE: 0,695 la start se transformă în 0,480 după 600 runde).	4.4.1, 5.3, 6.2.1, 6.2.2

2.4 Domeniul de aplicare al proiectului

Cerințele funcționale și non-funcționale stabilite anterior delimitează domeniul de aplicare al proiectului: aplicația mobilă Flutter cu suport Android și iOS cu recomandări de rute turistice, pornind de la pre-antrenarea globală și fiind îmbunătățite prin FL (simulat pe date sintetice cu obiective turistice din București); moderarea conținutului generat de turiști și localnici prin vot; autentificarea și gestionarea celor două tipuri de conturi (administrator și utilizator simplu). Rămâne în afara domeniului de aplicare personalizarea suplimentară la nivel de utilizator prin straturi separate ale modelului MLP (*personal head*), care nu ar fi trimise către server, rezultând într-o arhitectură de tip FedPer [3] - propusă ca direcție de lucrări viitoare, dar neimplementată în versiunea curentă. În plus, în afara domeniului se află și securitatea avansată la nivel de rețea, agregarea securizată (SecAgg [5]), incompatibilă cu FL-ul asincron utilizat, și validarea în producție și colectarea de date reale de interacțiune.

Având stabilite cerințele și limitele proiectului, Capitolul 3 trece în revistă soluțiile existente pentru explorare urbană (Google Maps, TripAdvisor, alte aplicații de tip *city-guide*) și realizează o analiză comparativă între abordările centralizate și cele bazate pe FL pentru recomandări.

3 SOLUȚII EXISTENTE

Înainte de proiectarea propriu-zisă a soluției, acest capitol analizează contextul în care se înscrie CultureQuest. Aplicațiile de explorare urbană existente oferă un grad bun de personalizare a recomandărilor, însă, acestea se bazează pe sisteme centralizate de colectare a datelor, de unde și comparația cu abordările bazate pe FL și cu stadiul actual al cercetării din domeniu, folosit pentru a poziționa CultureQuest în cadrul soluțiilor FL. Pe baza acestei analize, capitolul explică de ce o parte dintre cele mai cunoscute variante de algoritmi FL nu se potrivesc design-ului asincron al platformei și justifică restul tehnologiilor folosite în implementare.

3.1 Aplicații existente pentru explorare urbană

Platformele mobile de recomandare pentru turiști [6] oferă un anumit nivel de personalizare bazat pe istoricul de activitate al utilizatorului, stocat și procesat pe serverele furnizorului. **Google Maps** generează, de exemplu, recomandări pe baza istoricului de locații, a căutărilor și a evaluărilor asociate contului Google¹. **TripAdvisor** merge pe o logică asemănătoare, prin faptul că sugestiile de obiective și restaurante sunt adaptate pe baza recenziilor citite, a căutărilor anterioare și a locațiilor vizitate, tot prin contul de utilizator, procesat central². În ambele cazuri, personalizarea funcționează doar pentru că acest istoric de interacțiuni ajunge să fie analizat pe server, nu pe dispozitivul utilizatorului.

Pe lângă aceste platforme axate pe navigare, există și aplicații specializate în activități turistice legate de rezervări, precum **GetYourGuide**, care recomandă tururi cu ghizi, bilete și experiențe pe baza destinației căutate și a istoricului de căutări sau rezervări al utilizatorului, tot stocat și procesat central³. Spre deosebire de Google Maps sau TripAdvisor, accentul este pus pe activități plătite și predefinite, nu pe explorarea liberă a orașului, așa că nu există rute generate dinamic în funcție de poziția utilizatorului. Dincolo de aceste diferențe, toate trei se bazează pe același principiu - ML centralizat, antrenat pe date de la un număr mare de utilizatori.

O categorie specială o reprezintă aplicațiile care propun tururi audio ghidate, precum **izi.TRAVEL**, care oferă narațiuni geolocalizate și trasee audio predefinite pentru obiective culturale, în special muzee, din diverse orașe. Față de Google Maps sau TripAdvisor, accentul cade pe conținutul cultural structurat, nu pe navigare sau rezervări, iar traseele sunt pregătite de muzee, ghizi sau organizații certificate, iar aplicația pornește automat narațiunea pe măsură ce

¹Google, pagina oficială despre recomandările personalizate. <https://support.google.com/websearch/answer/17025248?hl=en> (accesat 19.06.2026).

²TripAdvisor, pagina oficială despre funcționarea recomandărilor/politica de confidențialitate. <https://tripadvisor.mediaroom.com/US-privacy-policy> (accesat 19.06.2026).

³GetYourGuide, pagina oficială despre cum funcționează recomandările și politica de confidențialitate. <https://www.getyourguide.com/c/privacy-policy> (accesat 19.06.2026).

utilizatorul ajunge în apropierea unui punct de interes⁴. Tururile pot fi descărcate pentru a funcționa în mod offline, dar lipsesc generarea dinamică de rute bazată pe profilul utilizatorului, personalizarea prin ML și gamificarea.

Gamificarea explorării bazate pe locația curentă este o componentă lipsă din toate aplicațiile prezentate mai sus, dar este acoperită parțial de **Geocaching**. Aici utilizatorii caută ascunzători fizice plasate de alți membri ai comunității, navighează la coordonatele GPS corespunzătoare și își înregistrează descoperirile. Ei pot strânge puncte și debloca realizări⁵. Cum conținutul este integral generat de utilizatori, Geocaching ajunge să fie cea mai apropiată alternativă pentru componenta de explorare gamificată. Cu toate acestea, recomandările personalizate tot nu sunt prezente nici în cadrul acestei aplicații, rutele nu sunt construite dinamic, partea de ML pentru recomandări lipsește și istoricul de activitate nu este stocat local, ci centralizat.

La polul opus se situează aplicațiile axate pe confidențialitate, precum **Organic Maps**, un proiect *open-source* bazat pe date de la OpenStreetMap, fără cont de utilizator și fără colectare de date⁶. Aici personalizarea nu există, iar toți utilizatorii dintr-o zonă văd același set de obiective, indiferent de interese, pentru că nu există niciun istoric din care acestea să poată fi accesate. Între cele două extreme - fie personalizare cu date colectate central, fie confidențialitate fără personalizare - se situează CultureQuest, unde scorul de personalizare este calculat prin FL, fără ca datele brute să părăsească dispozitivul. Toate aceste cazuri sunt adunate în **tabelul 3**.

Tabelul 3: Comparație între aplicații existente de explorare urbană și CultureQuest.

Aplicație	Rute turistice	Personalizare	Fără date pe server	Suport offline	Cost / acces	Gamificare
Google Maps	Navigație generală	Centralizată	Nu	Parțial	Gratuit / API plătit	Nu
TripAdvisor	Nu	Centralizată	Nu	Nu	Gratuit / API plătit	Limitat
GetYourGuide	Tururi predefinite	Limitată	Nu	Nu	Gratuit	Nu
izi.TRAVEL	Trasee audio predefinite	Nu	Nu	Parțial	Gratuit	Nu
Geocaching	Nu	Nu	Nu	Parțial	Freemium	Da
Organic Maps	Navigație generală	Nu	Da	Da	Gratuit, open source	Nu
CultureQuest	Dinamice, personalizate (FL)	Locală (FL)	Da	Da	Gratuit, open source	Da

⁴izi.TRAVEL, pagina oficială. <https://izi.travel/> (accesat 19.06.2026).

⁵Groundspeak Inc. (Geocaching HQ), termeni de utilizare și politică de confidențialitate. <https://www.geocaching.com/account/documents/privacypolicy> (accesat 19.06.2026).

⁶Organic Maps, pagina oficială - „no tracking, no data collection”. <https://organicmaps.app/privacy/> (accesat 19.06.2026).

3.2 Analiză comparativă între abordările centralizate și învățarea federată pentru recomandări

Sistemele de recomandare clasice se bazează, în general, pe filtrarea colaborativă, care recomandă unui utilizator ceea ce au apreciat alți utilizatori cu interese similare, și pe filtrarea bazată pe conținut, care recomandă obiective similare cu cele apreciate deja de același utilizator [2]. În ambele cazuri, determinarea similarităților are nevoie de o bază de date centralizată cu istoricul de activitate al clienților - exact modelul din spatele recomandărilor descrise în secțiunea anterioară pentru aplicațiile menționate. Avantajul este accesul la un volum mare de date eterogene, din care modelul învață tendințe generale utile chiar și pentru utilizatori noi, pe baza unor profiluri asemănătoare existente în sistem. Prețul acestui avantaj este, însă, tot cel discutat mai sus. Pentru ca modelul să poată face aceste calcule, istoricul de activitate al fiecărui utilizator trebuie să ajungă și să rămână pe server, sub controlul furnizorului.

FL schimbă direcția, astfel încât modelul ajunge la date, nu invers - antrenarea are loc local, pe dispozitivul fiecărui utilizator, iar pe server ajung doar actualizările rezultate ale modelului, nu istoricul de activitate în sine. Datele rămân astfel pe dispozitiv, iar confidențialitatea nu mai depinde de politicile furnizorului, ci de proiectarea protocolului de agregare. Diferența se vede cel mai clar la pornirea la rece (*cold-start*). O aplicație aflată la început nu are încă un model antrenat suficient din cauza unui număr mic de persoane care o folosesc, așa că recomandările vin dintr-un model global pre-antrenat și din caracteristicile bazate pe conținut ale obiectivelor, la fel ca în cazul centralizat - personalizarea prin FL se construiește abia pe măsură ce utilizatorii interacționează cu aplicația.

3.3 Învățare federată: Stadiul actual

FL a apărut ca direcție de cercetare odată cu FedAvg [13], ca răspuns direct la problema discutată în secțiunea anterioară - cum poate fi antrenat un model pe datele mai multor clienți, fără ca acestea să fie centralizate pe un server. De atunci, domeniul s-a diversificat, acoperind variante de agregare, metode DP, robustețe la clienți adversariali și scenarii de personalizare [7]. Pentru a situa CultureQuest în acest cadru, ne putem ghida după două axe de clasificare devenite standard în literatură: tipul de FL, în funcție de cum sunt distribuite datele între clienți, și modul de desfășurare, în funcție de cine sunt clienții și care este comportamentul lor.

Prima axă, propusă de Yang et al. [18], împarte FL în trei categorii:

- **Orizontal (HFL)** - clienții au același spațiu de caracteristici, dar mulțimi diferite de exemple, fiind situația tipică atunci când același tip de date este colectat de la utilizatori diferiți.

- **Vertical (VFL)** - clienții au în comun o parte din exemple, dar caracteristici diferite, ca în cazul unor organizații care dețin date diferite despre aceiași utilizatori.
- **FL cu transfer (FTL)** - pentru situațiile în care nici exemplele, nici caracteristicile nu se suprapun semnificativ.

CultureQuest se încadrează clar în categoria HFL. Fiecare telefon antrenează același model, pe același spațiu de 22 de caracteristici asociate obiectivelor turistice, diferența dintre clienți fiind doar istoricul propriu de interacțiuni.

A doua axă împarte FL în două moduri de desfășurare [7, 10]:

- **Cross-device** - clienții sunt un număr mare de dispozitive individuale, cu conexiuni nesigure și resurse limitate, fiecare contribuind cu o cantitate mică de date.
- **Cross-silo** - clienții sunt un număr mic de organizații sau servere, cu conexiuni stabile și seturi de date mari.

CultureQuest este, din această perspectivă, un sistem *cross-device*. Telefoanele utilizatorilor sunt conectate la internet intermitent, fiecare cu un volum mic de interacțiuni locale. Pentru aplicație, s-a optat, așadar, pentru agregare asincronă [17], tocmai pentru că acești clienți sunt intermitenți și nu este practic să se aștepte formarea unei cohorte complete la fiecare rundă de învățare. Este nevoie, prin urmare, și de mecanisme de robustețe la actualizări extreme [12]. **Figura 1** situează CultureQuest pe ambele axe.

	HFL	VFL	FTL
Cross-Device	CultureQuest		
Cross-Silo			

Figura 1: Poziționarea CultureQuest în taxonomia FL, pe axa tipului de FL (HFL/VFL/FTL) și axa modului de desfășurare (Cross-Device/Cross-Silo). Adaptat din Yang et al. 2019 [18] și Kairouz et al. 2021 [7].

3.4 Alternative respinse

FedAvg [13] este algoritmul de referință al FL. La fiecare rundă, un grup de K clienți antrenează local pe propriile date, iar serverul agregă actualizările prin medie ponderată. Eterogenitatea datelor dintre clienți este una dintre provocările cele mai notabile ale FL [10], iar o direcție importantă de cercetare propune algoritmi care corectează deriva (*drift*) dintre actualizările

locale ale clienților și modelul global. FedProx [11] adaugă funcției de cost locale un termen proximal, care penalizează distanța dintre ponderile actualizate și cele ale modelului global de la începutul runde. SCAFFOLD [8] introduce, pentru fiecare client, o variabilă de control care estimează cât de mult diferă actualizarea sa de media cohorței și corectează gradientul local pe baza acestei estimări. FedDyn [1] ajunge la un rezultat similar printr-un regularizator dinamic, recalculat la fiecare rundă pe baza istoricului actualizărilor clientului. Toate cele trei leagă actualizarea unui client de comportamentul celorlalți clienți selectați în aceeași rundă.

Această legătură nu are, însă, un corespondent în CultureQuest, unde agregarea se realizează prin FedAsync [17], care procesează, câte o actualizare pe rând, pe măsură ce sosește de la un client, fără să aștepte o cohortă. Antrenarea locală urmează, totuși, logica FedAvg (SGD pe interacțiunile locale ale utilizatorului). Fără cohorte, nu există actualizări paralele de reconciliat la final de rundă, dar nici conceptul de medie a grupului față de care un client să se abată - condiții pe care se bazează atât termenul proximal din FedProx, cât și variabilele de control din SCAFFOLD sau regularizatorul din FedDyn. Aplicat în acest context, un astfel de termen de corecție ar trage actualizarea locală către un punct de referință arbitrar, fără ca vreo garanție de convergență din literatură să mai fie valabilă, întrucât toate cele trei presupun, în demonstrațiile lor, runde sincrone cu mai mulți clienți. De aceea, design-ul asincron al CultureQuest are nevoie de altceva - un mecanism care reduce influența actualizărilor venite de la clienți cu un model local prea vechi. **Tabelul 4** rezumă această comparație.

Tabelul 4: Comparație între algoritmi FL principali și soluția adoptată de CultureQuest.

Algoritm	Sincron?	Mecanism	1 client/rundă?
FedAvg [13]	Da	Medie ponderată a cohorței de K clienți	Nu
FedAsync [17]	Nu	<i>Discount staleness</i> : $1/(1 + s)$	Da
FedProx [11]	Da	Termen proximal (penalizare față de modelul global)	Nu
SCAFFOLD [8]	Da	Variabilă de control per client	Nu
FedDyn [1]	Da	Regularizator dinamic per client	Nu
CultureQuest	Nu	FedAvg (doar pentru antrenarea locală) + FedAsync	Da

3.5 Tehnologiile utilizate și justificarea acestora

Alegerea tehnologiilor pentru CultureQuest a fost ghidată de câteva criterii comune, precum compatibilitatea *cross-platform* pentru aplicația mobilă, ecosistemul care se integrează natural cu componenta FL, cu inferența modelului și cu soluții *open-source* care pot fi găzduite local. În secțiunile următoare se descriu aceste alegeri, componentă cu componentă.

3.5.1 Flutter și Riverpod

Pentru aplicația mobilă, CultureQuest folosește Flutter⁷ și Riverpod⁸ pentru gestionarea stării. Flutter are avantajul de a permite o singură bază de cod Dart pentru Android și iOS, ceea ce înjumătățește efortul de mentenanță față de dezvoltarea nativă pe cele două platforme, fiind relevant, mai ales, pentru a nu duplica logica clientului FL și, totodată, pentru a nu îngreuna sincronizarea. Flutter compilează codul Dart direct în cod nativ ARM, fără un *bridge* către componentele native, eliminându-se *overhead-ul* de comunicare regăsit în React Native la fiecare apel nativ⁹. În ceea ce privește Riverpod, acesta oferă un model de gestionare a stării declarativ și testabil. Este, prin urmare, potrivit pentru o aplicație în care starea ecranului curent trebuie sincronizată cu procese care rulează în fundal, precum antrenarea locală a modelului.

3.5.2 FastAPI, MongoDB și Redis

Pe partea de *backend*, CultureQuest folosește FastAPI¹⁰, MongoDB¹¹ și Redis¹². FastAPI este scris în Python, ceea ce permite integrarea directă cu ecosistemul NumPy folosit la antrenare și la inferența MLP, fără strat de conversie între limbaje. Mai mult, suportul nativ pentru operații asincrone¹³ se potrivește cu un număr mare de clienți conectați intermitent. Alegerea MongoDB, adică a unei baze de date non-relaționale, se justifică prin stocarea de date eterogene cu câmpuri diferite - obiective, *quest-uri*, povești, recenzii - care într-o bază de date relațională precum PostgreSQL ar necesita tabele de *join* suplimentare. Interogările de proximitate sunt acoperite de indexul 2dsphere nativ al MongoDB și, deci, nu este nevoie de o extensie precum PostGIS, ale cărei capabilități GIS avansate nu sunt necesare când OSRM se ocupă deja de partea de rutare. Redis, la rândul lui, acoperă starea nepersistentă a procesului FL, cu o latență de sub 1 ms pentru operații în memorie și cu suport pentru SET NX PX atomic - pentru lock-ul de agregare (`f1:agg_lock`) care protejează ciclul read-aggregate-write.

⁷Documentația oficială Flutter. <https://flutter.dev> (accesat 19.06.2026).

⁸Documentația oficială Riverpod. <https://riverpod.dev> (accesat 19.06.2026).

⁹Flutter, documentație privind performanța și arhitectura de compilare. <https://docs.flutter.dev/perf> (accesat 19.06.2026).

¹⁰Documentația oficială FastAPI. <https://fastapi.tiangolo.com> (accesat 19.06.2026).

¹¹Documentația oficială MongoDB. <https://www.mongodb.com/docs/> (accesat 19.06.2026).

¹²Documentația oficială Redis. <https://redis.io/docs/> (accesat 19.06.2026).

¹³FastAPI, documentație privind operațiile asincrone. <https://fastapi.tiangolo.com/async/> (accesat 19.06.2026).

3.5.3 Motorul de rutare OSRM

Funcționalitatea de rutare se bazează pe OSRM (*Open Source Routing Machine*)¹⁴, un motor *open-source* bazat pe date OpenStreetMap. Poate fi găzduit local, pe același server cu restul serviciilor de *backend*, fără dependență de un API extern plătit (Google Maps Direction API facturează, de exemplu, per cerere¹⁵). OSRM calculează rute pe rețeaua de drumuri folosind Contraction Hierarchies, și astfel, răspunde în câteva zeci de milisecunde pentru o rută obișnuită ca distanță în perimetrul unui oraș¹⁶ - suficient de rapid pentru a fi folosit de utilizator pe parcursul unei rute.

3.5.4 Perceptron multi-strat

Modelul de recomandare al CultureQuest este un MLP. Arhitectura este simplă: trei straturi complet conectate (22-32-16-1), pornind de la un vector de 22 de caracteristici asociate obiectivelor turistice și culturale. Pe dispozitiv, modelul nu este folosit doar pentru antrenarea locală, ci și pentru afișare - de fiecare dată când utilizatorul deschide pagina cu informații a unui obiectiv turistic, aplicația face o nouă trecere *forward pass* prin model, ca să calculeze scorul de potrivire afișat pentru turist. Tocmai pentru că rulează la fiecare interacțiune, modelul trebuie să fie o implementare proprie, cât mai ușoară și fără dependențe externe.

Alternativele cunoscute pentru rularea unui model de rețea neuronală pe un dispozitiv sunt TFLite¹⁷ și PyTorch Mobile¹⁸, ambele *runtime-uri* dedicate, cu dependențe native pentru fiecare platformă. TFLite adaugă $\approx 1,3$ MB¹⁹ la APK (arm64), PyTorch Mobile ≈ 30 MB²⁰, fără avantaje vizibile pentru un model de 5 KB. Implementarea în Dart rămâne, deci, mai mică, fără dependențe de *runtime* suplimentare, și compatibilă nativ cu Flutter, după cum reiese și din **tabelul 5**.

¹⁴Documentația oficială OSRM. <https://project-osrm.org> (accesat 19.06.2026).

¹⁵Google Maps Platform, prețuri Directions API. <https://developers.google.com/maps/documentation/directions/usage-and-billing> (accesat 19.06.2026).

¹⁶Luxen, D., & Vetter, C. (2011). Real-time routing with OpenStreetMap data. *Proceedings of the 19th ACM SIGSPATIAL*. <https://doi.org/10.1145/2093973.2094062>

¹⁷Documentația oficială TensorFlow Lite. <https://www.tensorflow.org/lite> (accesat 19.06.2026).

¹⁸PyTorch Mobile este parțial înlocuit de ExecuTorch; documentația oficială ExecuTorch. <https://docs.pytorch.org/executorch/stable/index.html> (accesat 19.06.2026).

¹⁹Google. TFLite: reduce binary size. https://www.tensorflow.org/lite/guide/reduce_binary_size (accesat 19.06.2026).

²⁰PyTorch. PyTorch Mobile. <https://pytorch.org/mobile/home/> (accesat 19.06.2026).

Tabelul 5: Comparație între implementarea MLP proprie și runtime-urile dedicate (TFLite, PyTorch Mobile) pentru inferență pe dispozitiv.

Implementare	Parametri / Dimensiune model	Runtime adăugat la APK	Dependențe native	Timp inferență
MLP propriu (Dart)	1.281 param. (22-32-16-1), ≈ 5 KB (valori în format float32)	0 MB (compilat nativ)	Niciuna	$\approx 40 \mu s$
TFLite	depinde de model	$\approx 1,3$ MB (arm64)	Bibliotecă pentru Android/iOS	< 1 ms
PyTorch Mobile	depinde de model	≈ 30 MB (arm64)	Bibliotecă pentru Android/iOS	< 1 ms

4 SOLUȚIA PROPUȘĂ

Acest capitol detaliază proiectarea CultureQuest, cu suportul a câtorva diagrame, tabele și ecuații care justifică deciziile de design. Pornește de la o privire de ansamblu asupra arhitecturii, după care urmează aplicația mobilă și serviciile *backend*, fiecare cu propria diagramă de structură. Cea mai mare parte a capitolului este dedicată proiectării sistemului FL (4.4). Urmează fluxul de generare a rutelor, elementele de gamificare și harta cu navigarea, iar în final capitolul se încheie cu arhitectura de confidențialitate, care descrie deciziile de proiectare legate de separarea datelor între dispozitiv și server.

4.1 Prezentare generală a sistemului

La nivel general, CultureQuest este format din patru componente: aplicația mobilă, scrisă în Flutter, un *backend* FastAPI, o bază de date MongoDB pentru date persistente și o instanță Redis pentru starea nepersistentă a procesului FL. **Figura 2** arată comunicarea dintre ele. Aplicația mobilă este singurul punct de legătură dintre utilizator și sistem - toate cererile trec prin *backend*, care, în funcție de tipul de date, citește sau scrie în MongoDB sau în Redis.

Diagrama evidențiază trei fluxuri principale, pe care le-am numerotat cu (1), (2) și (3). Primul (1), cererea de hartă și rute, funcționează astfel: aplicația mobilă trimite poziția curentă, *backend-ul* interoghează baza de date pentru obiectivele din apropiere, le oferă un scor prin intermediul modelului FL global ținut în Redis și calculează, pe baza scorurilor, o rută pe care o trimite înapoi. Al doilea (2), agregarea FL, funcționează similar: dispozitivul trimite actualizarea modelului antrenat local, *backend-ul* o agregă cu ajutorul stării ținute în Redis și trimite înapoi modelul global actualizat. Recenziile, poveștile și *quest-urile* adăugate de utilizatori formează al treilea flux (3), unde acestea trec printr-un pas de moderare înainte să fie salvate în MongoDB.

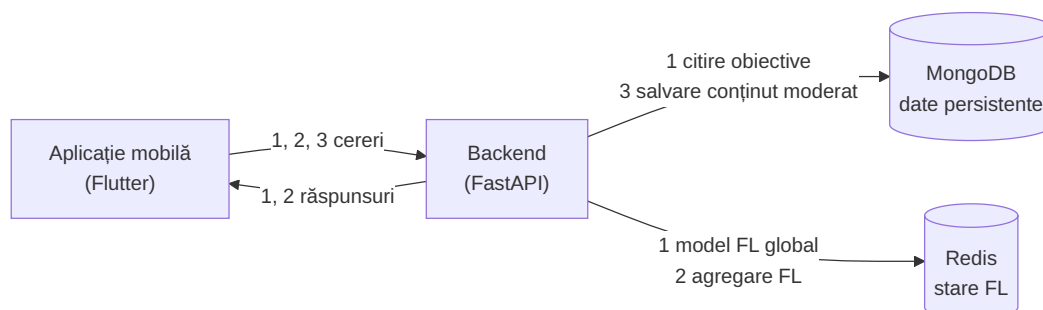


Figura 2: Arhitectura generală a sistemului CultureQuest: aplicația mobilă comunică exclusiv cu *backend-ul*, care citește sau scrie în MongoDB și Redis.

4.2 Arhitectura aplicației mobile

Aplicația mobilă este organizată pe module de tip *feature*, unde fiecare funcționalitate principală este asociată unui director propriu cu ecrane și provideri, fără dependențe directe față de alte module. Cele șapte module sunt *map* - pentru hartă și obiectivele de pe aceasta, dar și pentru generare de rute, *auth* - pentru autentificare, *federated* - pentru clientul FL, *onboarding* - pentru alegerea intereselor de către utilizator la instalarea aplicației, *profile* - pentru profilul utilizatorului și statusul procesului FL, *proximity* - pentru partea *geofencing* și detectarea proximității față de obiective și *quest* - pentru sistemul de misiuni.

Gestionarea stării intră în responsabilitatea Riverpod, care leagă modulele între ele prin provideri declarativi. Aceștia sunt disponibili de la orice nivel al arborelui de *widget-uri*, rezultând avantajul că datele nu mai sunt transmise manual prin constructori. **Figura 3** ilustrează arhitectura în trei straturi. Modulele de tip *feature* folosesc provideri Riverpod specializați, care la rândul lor delegă atât cererile către *backend*, cât și operațiile de stocare locală altor servicii dedicate. Providerii partajați arată dependențele funcționale dintre module: *fl_provider* este utilizat de *map* (pentru scorul de potrivire afișat în cadrul unui obiectiv), de *federated* (pentru statusul agregării FL) și de *profile* (pentru statusul antrenării locale). De asemenea, *proximity_provider* este partajat de *proximity* și *quest*, cel din urmă depinzând de evenimentele de *geofencing* pentru a permite completarea *quest-urilor* disponibile pentru un obiectiv.

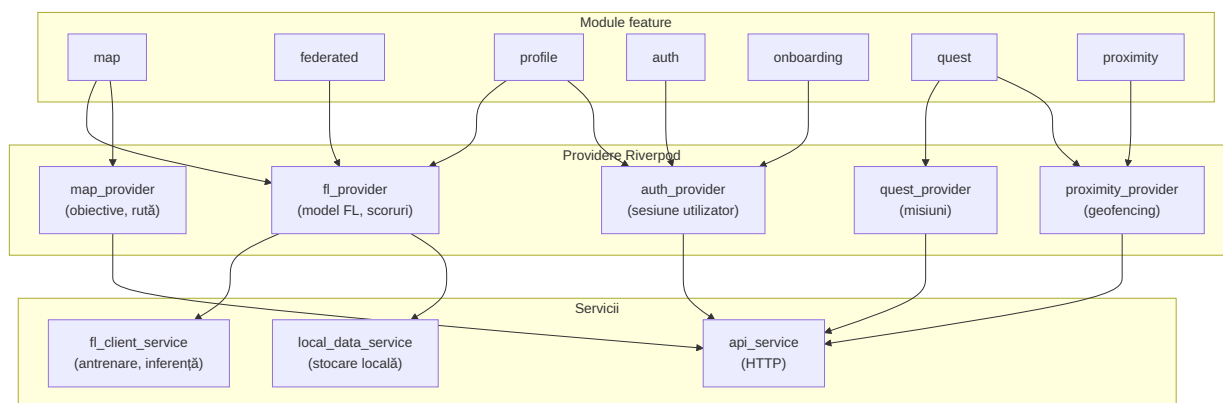


Figura 3: Arhitectura aplicației mobile CultureQuest: modulele de tip *feature* folosesc *provideri* Riverpod partajate.

4.3 Arhitectura serviciilor *backend*

Backend-ul aplicației este implementat cu FastAPI și structurat pe domenii de funcționalitate. Fiecare colecție MongoDB are un grup de rute dedicat și un serviciu de logică de business asociat. Grupurile de rute sunt împărțite în felul următor: un grup pentru autentificarea utilizatorilor (*auth*, cu înregistrare și autentificare pe bază de token JWT), obiectivele turistice și

culturale (*landmarks*), misiunile (*quests*), rutele turistice (*routes*), evenimentele (*events*), recenziiile (*comments*), validarea comunitară a conținutului (*community*) și schimbul de ponderi FL cu clienții (*federated*).

Modelul de date - a cărei schemă completă este ilustrată în **figura 8** din Anexe - este construit în jurul colecțiilor **USERS** și **LANDMARKS**: USERS stochează profilul utilizatorului, format din credențialele acestuia și lista de interese pe care le poate selecta, și care intră ulterior în construirea vectorului de caracteristici pentru modelul FL, iar LANDMARKS reprezintă obiectivele care au drept caracteristică principală locația în format GeoJSON indexată 2dsphere, necesară interogărilor de proximitate. De colecția LANDMARKS sunt legate direct **QUESTS**, **EVENTS**, **COMMENTS**, **STORIES** și **VISITS**, dar și **ROUTES** pentru stocharea rutelor ca secvențe ordonate de referințe către LANDMARKS (fie rute globale pre-generate, fie generate de utilizatorii care doresc să parcurgă un traseu cu obiectivele turistice din apropiere).

Conținutul generat de utilizatori trece printr-un flux de moderare înainte de publicare. *Questurile* propuse și poveștile sunt create cu statusul *pending* și devin disponibile altor utilizatori doar după aprobarea de către un administrator sau de către cinci alți utilizatori. Comentariile sunt, însă, publicate imediat, dar pot fi marcate automat prin câmpul *flagged* de modulul de moderare a conținutului, acestea ajungând să fie vizibile conturilor de tip administrator pentru a fi revizuite manual.

4.4 Proiectarea sistemului de învățare federată

Sistemul FL din CultureQuest este proiectat să asocieze un scor de potrivire fiecărui obiectiv și să personalizeze generarea rutelor fără ca datele de interacțiune să părăsească dispozitivul utilizatorului. Fluxul FL este structurat în șase componente: un model MLP de dimensiuni reduse care rulează pe dispozitiv, o procedură de antrenare locală bazată pe FedAvg, un mecanism de agregare asincronă cu reducere pe bază de vechime conform FedAsync, limitarea normei actualizărilor agregate pentru ca modelul să nu fie afectat de contribuțiile extreme, adăugarea de zgomot Gaussian pentru garantarea DP, dar și un mecanism de control al concurenței bazat pe Redis.

4.4.1 Arhitectura modelului

Modelul folosit pentru personalizarea din cadrul aplicației este un MLP cu arhitectura $22 \rightarrow 32 \rightarrow 16 \rightarrow 1$, model ilustrat în **figura 4**, în care se descrie faptul că straturile ascunse folosesc funcția de activare ReLU, pe când stratul de ieșire aplică funcția sigmoid, producând un scor continuu în $[0, 1]$ care determină gradul de potrivire dintre profilul utilizatorului și un obiectiv dat. Modelul însumează 1.281 de parametri, acest număr mic de parametri fiind ales deliberat pentru a menține costul computațional redus pe dispozitiv. Acesta rulează atât în

antrenare, cât și la fiecare deschidere a fișei unui obiectiv.

Vectorul de intrare de 22 de dimensiuni grupează caracteristicile în patru categorii, iar fiecare caracteristică este detaliată în **tabelul 6**. Primele 14 dimensiuni codifică profilul de interese al utilizatorului și tipul obiectivului în format *one-hot*. Contextul temporal (dimensiunile 14-16) stabilește disponibilitatea obiectivului la momentul interogării, dacă la momentul actual este weekend și mai include ora normalizată la $[0, 1]$. Contextul spațial și de rută (dimensiunile 17-21) includ rangul relativ de distanță față de alte posibile obiective care pot fi alese pentru rută, scorul de potrivire a intereselor și trei caracteristici care sunt active doar în etapa de generare a rutelor.

Același model rulează pentru fiecare locație în parte, cu un vector de caracteristici construit specific acesteia, scorul rezultat fiind convertit într-o categorie de potrivire afișată pe fișa obiectivului - **scăzut** pentru scoruri sub 0,4, **mediu** pentru intervalul $[0,4, 0,7)$ și **ridicat** pentru scoruri mai mari sau egale decât 0,7.

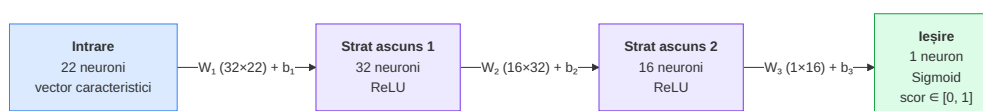


Figura 4: Arhitectura modelului MLP utilizat pentru personalizare.

Tabelul 6: Vectorul de caracteristici de intrare al modelului FL (22 dimensiuni).

Dim.	Caracteristică	Tip	Descriere
0–5	Interesele utilizatorului	one-hot	artă, arhitectură, istorie, gastronomie, natură, muzică
6–13	Tipul obiectivului	one-hot	muzeu, monument, parc, galerie, restaurant, piață, clădire, altele
14	isOpen	binar	obiectivul este deschis
15	isWeekend	binar	ziua curentă este sâmbătă sau duminică
16	oră normalizată	float $[0, 1]$	ora curentă împărțită la 24
17	relativeDistanceRank	float $[0, 1]$	proporția candidaților mai apropiați; 0 = cel mai apropiat
18	interestMatchScore	float $[0, 1]$	proporția categoriilor comune între utilizator și obiectiv
19	isPartOfRoute	binar	obiectivul face parte din ruta curentă
20	routeStopNormalized	float $[0, 1]$	poziția opririi în rută; 0 = prima oprire, 1 = ultima
21	routeLengthNormalized	float $[0, 1]$	lungimea rutei normalizată la maxim 5 opriri

4.4.2 Antrenarea locală

Procedura de antrenare locală este implementată conform protocolului clientului din FedAvg [13]. Dispozitivul antrenează modelul pe un buffer local de interacțiuni și trimite ulterior către server doar variația ponderilor față de modelul global. O rundă de antrenare se declanșează automat după strângerea a 15 interacțiuni în buffer, pragul ales asigurând un set minim divers de exemple înainte de actualizare. Runda poate fi inițiată și manual, de către utilizator, dacă au fost strânse cel puțin 8 interacțiuni cu obiectivele, pragul ales fiind mai mic aici, deoarece utilizatorul alege conștient să trimită actualizările către server. Fiecare rundă presupune rularea a 5 epoci complete peste întregul set local, număr testat în literatura de bază FL [13], aducând un echilibru între reducerea numărului de runde de învățare și limitarea derivei locale a modelului față de cel global.

Optimizarea se realizează prin SGD cu rata de învățare $\eta = 0,01$, care este aplicată pentru fiecare eșantion, valoarea conservatoare prevenind divergența în prezența zgomotului Gaussian adăugat în pasul de DP. Pentru funcția de pierdere se folosește eroarea medie pătratică (MSE, *Mean Squared Error*), cu formula actualizărilor ilustrată în **ecuația 1**. Alegerea MSE față de BCE este motivată de natura etichetelor de antrenare: acestea sunt valori continue în $[0, 1]$, nu valori de 0 sau 1 discrete, iar gradientii MSE sunt mărginiți, pe când BCE are gradienti nemărginiți pe măsură ce predicția se apropie de 0 sau 1, ceea ce poate destabiliza antrenarea în prezența zgomotului Gaussian adăugat în pasul de DP; însă, simularea evaluează convergența cu BCE, care este o metrică mai sensibilă pentru modelul cu ieșire sigmoid, deoarece scara logaritmică penalizează mai sever predicțiile confident greșite față de eroarea pătratică. Gradientul acestor actualizări este limitat la norma L2 unitară, iar dacă norma depășește 1, gradientul este scalat proporțional. Procesul este diferit de limitarea variației ponderilor aplicată ulterior în pasul de DP.

$$w_{t+1} = w_t - \eta \cdot \frac{g_t}{\max(1, \|g_t\|_2)} \quad (1)$$

unde w_t reprezintă ponderile la pasul t , $\eta = 0,01$ este rata de învățare, g_t este gradientul erorii medii pătratice calculat per eșantion, iar $\|g_t\|_2$ este norma sa L2.

4.4.3 Agregarea asincronă și reducerea în funcție de vechime

Serverul agregă imediat fiecare actualizare primită, fără a aștepta alți clienți, indiferent dacă este vorba doar de un subset dintre ei, procesul de învățare de pe dispozitiv și apoi agregarea reprezentând o rundă. Clientul trimite către server numărul runde la care a descărcat modelul global (`client_round`), iar *staleness-ul* reprezintă câte runde au trecut de când clientul a primit modelul actualizat. Factorul de reducere [17], prezentat în **ecuația 2**, scalează numărul efectiv de eșantioane.

Modelul global este ponderat cu de 4 ori mai multe eşantioane virtuale, astfel încât contribuția unui client să nu poată depăși 20% din agregare. Această limitare compensează absența rotunjirii sincrone: în FedAvg clasic, diluarea contribuțiilor individuale apare natural din numărul de clienți participanți simultan, iar în varianta asincronă, unde clienții sunt procesați pe rând, același efect trebuie impus explicit. Am ales și un prag minim de 20 de eşantioane virtuale care asigură că, pentru clienții cu discount de *staleness* extrem, procentul de contribuție este cel mult $\approx 5\%$.

Funcția de reducere este ilustrată în **figura 5**, unde se observă că la *staleness*=0 clientul contribuie integral, la *staleness*=1 în proporție de 50%, iar la *staleness*=9 în proporție de doar 10%. Alegerea algoritmului se justifică prin următorul caz: dacă doi - sau mai mulți - clienți descarcă modelul global în același timp și unul termină antrenarea mai repede, serverul ajunge la runda următoare înainte ca al doilea să trimită actualizarea. Prin urmare, discountul penalizează proporțional contribuția celui rămas în urmă cu varianta modelului global.

$$\text{discount}(s) = \frac{1}{1+s}, \quad n_{\text{eff}} = \max(1, \lfloor n \cdot \text{discount}(s) \rfloor) \quad (2)$$

unde $s = \text{current_round} - \text{client_round}$ este vechimea actualizării, n este numărul de eşantioane locale ale clientului, iar n_{eff} este numărul efectiv folosit la agregare.

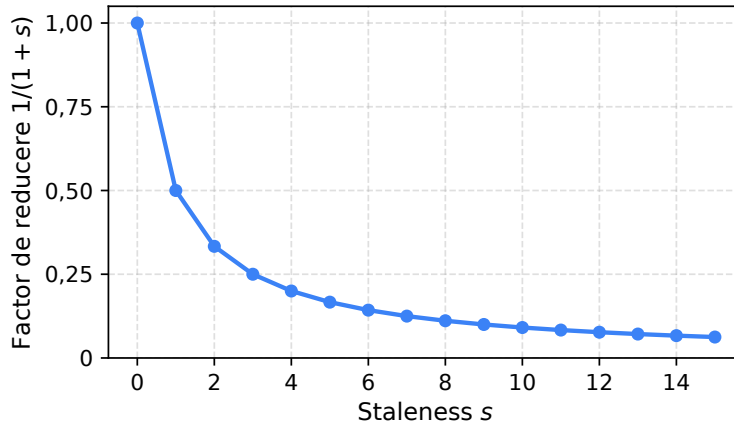


Figura 5: Factorul de reducere FedAsync în funcție de *staleness*.

4.4.4 Limitarea gradientilor

Înainte de agregare, serverul calculează variația ponderilor față de modelul global, denumită și delta. Dacă norma L2 a deltei depășește pragul $\tau = 1$, aceasta este rescalată proporțional conform **ecuației 3** [14], după care actualizarea rescalată este trecută prin agregarea ponderată FedAvg. Pentru un client care antrenează în mod normal, delta are o normă mică și factorul de scalare este 1, deci, actualizarea trece nemodificată. Limitarea intervine doar în cazul în care delta este foarte mare față de ce ar produce o antrenare obișnuită.

Rolul principal al acestui tip de limitare este de a reduce influența actualizărilor extreme. Un client malițios care evaluează totul adversarial [12] produce o deltă cu normă mare, dar aceasta este redusă la τ înainte de agregare, limitând influența sa pentru fiecare coordonată de pondere, însă, limitarea deltei este și o condiție necesară pentru garantarea DP din secțiunea următoare.

$$\tilde{\delta} = \delta \cdot \min\left(1, \frac{\tau}{\|\delta\|_2}\right) \quad (3)$$

unde $\delta = w_{\text{client}} - w_{\text{global}}$ este variația ponderilor față de modelul global, $\|\delta\|_2$ este norma sa L2, iar $\tau = 1$ este pragul de limitare.

4.4.5 Confidențialitate diferențială

Perturbarea prin zgomot nu se face central, zgomotul Gaussian fiind adăugat pe dispozitivul clientului, nu pe server, conform mecanismului NbAFL (*Noising before Model Aggregation Federated Learning*) [16]. Imediat după etapa de antrenare locală, fiecare coordonată a deltei este limitată la intervalul $[-\tau, \tau]$, și abia peste valorile rezultate se suprapune zgomotul. În felul acesta, la server ajung ponderile cu zgomotul deja aplicat, din care actualizarea reală nu mai poate fi reconstituită niciodată.

Limitarea făcută coordonată cu coordonată la acest pas nu trebuie confundată cu limitarea normei L2 discutată la secțiunea anterioară: aceea tratează delta ca pe un singur vector și are rolul de a apăra modelul de actualizările adversariale [12], pe când cea de față ține sensibilitatea mărginită, condiție care face ca zgomotul adăugat să aibă efect real asupra confidențialității [15]. Cât despre intensitatea lui, valoarea aleasă (0,1) reprezintă un echilibru asumat între confidențialitate și utilitatea modelului, pe care îl analizez în secțiunea de evaluare (Capitolul 6).

$$\tilde{w} = w_{\text{global}} + \text{clip}(\delta, \tau) + \mathcal{N}(0, (\sigma \cdot \tau)^2) \quad (4)$$

unde $\delta = w_{\text{trained}} - w_{\text{global}}$ este variația ponderilor, $\tau = 1$ este pragul de limitare, iar $\sigma = 0,1$ este multiplicatorul de zgomot.

4.4.6 Controlul concurenței

Citirea modelului global, agregarea și rescrierea lui nu alcătuiesc o operație atomică în mod implicit. Dacă doi clienți trimit actualizări în același timp, se poate ajunge ca ambii să citească modelul la aceeași versiune, să-și calculeze delta fiecare, independent, dar să-și suprascrie reciproc rezultatul agregării. Pentru a preveni acest caz, am implementat pentru server un

lock Redis bazat pe SET NX PX: un token unic care este scris în Redis sub cheia `fl:agg_lock`, doar dacă aceasta nu există deja, ceea ce asigură că un singur client poate intra în etapa de agregare la un moment dat. Eliberarea *lock-ului* se face printr-un script Lua rulat atomic, care mai întâi verifică că token-ul aparține clientului curent pentru a evita ștergerea eronată a unui *lock* deținut de alt client.

Dacă *lock-ul* nu poate fi obținut, serverul mai încearcă de 9 ori, la intervale de câte 0,3 secunde - fereastra totală de 2,7 secunde acoperă scenariul a 10 cereri simultane, unde latența p95 a agregării este de 2,44 secunde (*load test*, Capitolul 6) - iar dacă tot nu reușește, întoarce o eroare. *Lock-ul* expiră de la sine după 5 secunde (valoare aleasă să depășească p95 la 10 cereri), în cazul în care serverul devine indisponibil în mijlocul agregării și *lock-ul* rămâne blocat. Am redat fluxul întreg în **figura 6**.

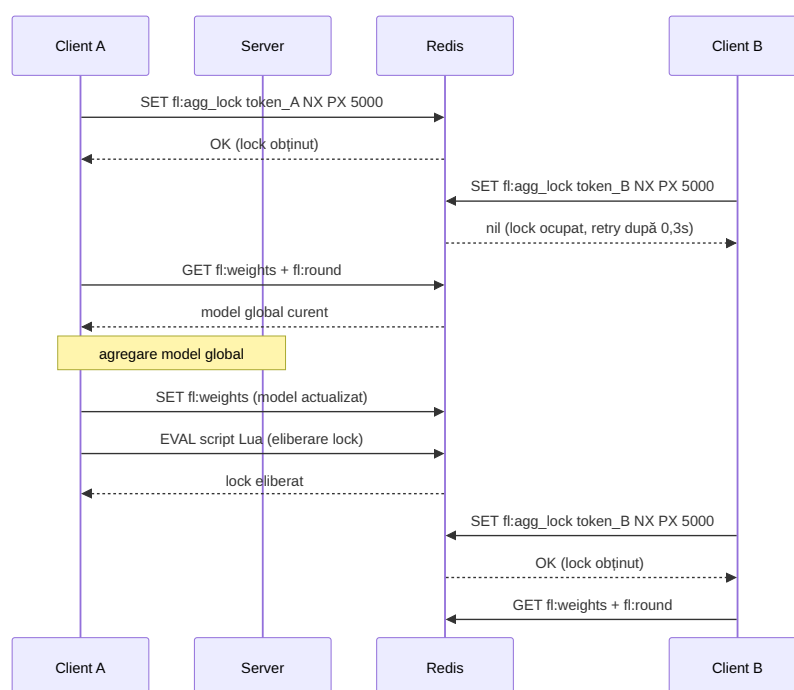


Figura 6: Controlul concurenței prin lock Redis.

4.5 Fluxul de generare a rutelor

Generarea unei rute cuprinde, mai întâi, locația curentă a utilizatorului, rezultând, în final, un traseu ordonat de obiective din proximitate, alcătuit astfel încât să respecte preferințele legate de interese și un buget alocat de maxim 300 de minute (o sesiune tipică de jumătate de zi). Calculul rulează pe server, în `route_service.py`, și este schițat în **figura 7**. În prima etapă se strâng toate obiectivele turistice aflate pe o rază de 10 km, fiind o distanță care acoperă perimetrul centrului unui oraș și corespunde aproximativ cu două ore de deplasare pe jos. Apoi, li se atribuie un scor inițial care combină scorul FL (în proporție de 80%) și

proximitatea față de locația de start (în proporție de 20%). Ponderea mare a FL se justifică prin faptul că vectorul de caracteristici include deja rangul relativ de distanță, aceasta fiind deja luată în considerare indirect, pe când componenta de proximitate de 20% previne situația în care un obiectiv îndepărtat cu scor mare din punct de vedere FL este ales primul. Dacă modelul global nu este disponibil în momentul respectiv, locul scorului FL este preluat de o fracție de potrivire a intereselor, fiind dedusă direct din categoriile obiectivului. Primele 10 locații - dacă există în apropiere - cu cel mai mare scor rămân drept candidați.

Am ales ca traseul să fie construit cu ajutorul unui algoritm greedy de tip vecin cel mai apropiat, adecvat pentru numărul mic de opriri dintr-o rută turistică (cel mult 5), deoarece o abordare exactă ar avea un cost computațional mai mare fără beneficii vizibile. La fiecare pas, se alege obiectivul cel mai aproape de poziția curentă, însă, cu o corecție care „scurtează” distanța față de candidații ale căror categorii se potrivesc intereselor utilizatorului. Pentru fiecare oprire se estimează și o durată de vizitare, durata care pleacă de la un timp de bază specific tipului de obiectiv, estimat pe baza comportamentului tipic al turiștilor sau localnicilor (de la 15 minute pentru o piață publică până la 70 de minute pentru un muzeu). Mai mult, este scalată între 75% și 125% în funcție de scorul FL al opririi. Intervalul de $\pm 25\%$ este suficient de mare pentru a reflecta preferința utilizatorului, dar nu atât de extrem încât să distorsioneze în mare măsură bugetul total de timp al rutei.

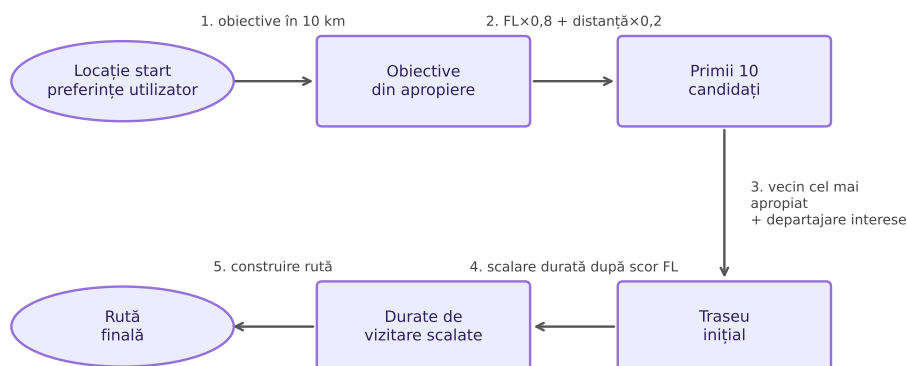


Figura 7: Fluxul de generare a rutelor.

4.6 Proiectarea elementelor de gamificare

Gamificarea se sprijină pe sistemul de *quest-uri* asociate obiectivelor turistice și culturale - sarcini scurte pe care utilizatorul le poate accepta și duce la capăt doar la fața locului, fiind răsplătit cu puncte la completare. Un *quest* poate fi adăugat atât de un administrator sau de o persoană care a creat obiectivul asociat pe hartă (și va fi disponibil imediat altor utilizatori),

cât și de o persoană oarecare (în acest caz fiind nevoie ori de aprobarea unui administrator, ori de votul a cinci utilizatori fără rol de administrator). Pragul de cinci voturi este suficient de mic pentru a fi atins într-o comunitate activă, și, în același timp, suficient de mare pentru a aduce mai mulți utilizatori independenți laolaltă, iar voturile sunt anonime, păstrându-se principiul de confidențialitate al aplicației.

Fiecare interacțiune cu un obiectiv lasă în urmă un scor de *engagement* al turistului cu obiectivul, care reprezintă ținta de antrenament a modelului FL pentru acea interacțiune. Valoarea scorului crește odată cu „intensitatea interacțiunii”: deschiderea fișei unui obiectiv generează valoarea 0,3, adăugarea la rută a obiectivului sau un răspuns greșit la un *quest* aduce un scor de 0,5, iar un răspuns corect 0,9. Am ales valorile empiric, pentru a reprezenta o bună corelație cu dorința de implicare în a descoperi mai mult despre obiectivul curent. Așadar, scorul maxim generat prin aceste interacțiuni va deveni scorul asociat obiectivului, și va fi folosit pentru etapa de *backpropagation*. Evaluările cu stele sunt convertite după formula $n/5$, producând valori între 0,2 și 1 în funcție de numărul de stele lăsat de utilizator în cadrul recenziei, dar, în acest caz, valoarea rezultată suprascrie orice valoare produsă de interacțiunile anterioare, chiar dacă ar fi mai mare. Setul complet de etichete este redat în **tabelul 7**.

Tabelul 7: Etichete de *engagement*.

Eveniment	Etichetă
Fișa obiectivului deschisă	0,3
Obiectiv adăugat la rută	0,5
<i>Quest</i> : răspuns incorect	0,5
Navigare către obiectiv pornită	0,6
<i>Quest</i> propus de utilizator	0,80
Poveste trimisă	0,85
<i>Quest</i> : răspuns corect	0,9
Evaluare cu n stele	$n/5$

4.7 Harta și navigarea

Harta aplicației integrează tehnica de *geofencing*, implementată manual pe baza fluxului de poziții GPS furnizat de pachetul *geolocator*¹. De fiecare dată când poziția se actualizează, serviciul de proximitate recalculează distanțele până la obiectivele din jur și declanșează un eveniment de intrare sau ieșire în momentul în care utilizatorul traversează raza unui obiectiv. Lista obiectivelor monitorizate nu este, însă, reîncărcată la nesfârșit, ci abia după ce utilizatorul s-a deplasat cu mai mult de 200 de metri. Orientarea hărții este gestionată, la rândul ei,

¹Pachet Flutter pentru accesul la localizare GPS. <https://pub.dev/packages/geolocator> (accesat 19.06.2026)

prin două mecanisme: în repaus, pachetul *flutter_compass*² rotește harta după busola magnetometrică și ignoră schimbările sub 2° pentru a nu prelua zgomotul senzorului; în modul de navigare activă, harta se rotește după direcția de deplasare raportată de GPS.

4.8 Arhitectura de confidențialitate

Arhitectura de confidențialitate a CultureQuest pornește de la ideea că datele trebuie separate explicit în funcție de locul de stocare. Tot ce ține de comportamentul concret al turistului - obiectivele vizitate, rating-urile date fiecărui obiectiv în parte, *quest-urile* finalizate și rutele salvate - rămân local, pe telefon, prin *SharedPreferences*, în directorul privat al aplicației, în așa fel încât alte aplicații să nu aibă acces la date, fiind o responsabilitate a *sandbox-ului* Android/iOS, iar spre server vin doar informații agregate sau anonimizate. Concret, se incrementează un contor global de vizite pentru fiecare obiectiv în locul primirii identității vizitatorului, respectiv se modifică suma și numărul de rating-uri în locul evaluării de către vizitator. Pe server, totul se rezumă la a se păstra doar datele necesare funcționării aplicației: numele complet, punctajul total, numărul de *quest-uri* completate și interesele selectate. Aceeași logică se aplică și componentei FL: serverul primește doar delta-ul ponderilor obținut după antrenamentul local, peste care s-a adăugat deja zgomot Gaussian. Distribuția completă este redată în **tabelul 8**, iar modul în care sunt implementate mecanismele de confidențialitate este detaliat în Capitolul 5.

Arhitectura actuală are, însă, și o limitare: serverul primește delta-ul fiecărui client în mod individual, așa că se pot distinge, în principiu, contribuțiile individuale. Protocolul de agregare securizată [5] ar elimina acest risc, făcând ca serverul să vadă doar suma actualizărilor, nu și delta-urile individuale. Numai că SecAgg este incompatibil cu configurația asincronă pe care se bazează platforma. Protocolul cere ca un set fix de clienți să fie activi în același timp pentru schimbul de chei criptografice, ceea ce contravine modelului FedAsync în care clienții se conectează independent, fiecare la momentul lui. Mai mult, discountul de *staleness* aplicat pentru fiecare client la agregare are nevoie de acces la delta-urile individuale, nu doar la suma lor.

²Pachet Flutter pentru citirea busolei magnetometrice. https://pub.dev/packages/flutter_compass (accesat 19.06.2026)

Tabelul 8: Distribuția datelor în funcție de locul de stocare.

Date locale (SharedPreferences)	Date pe server	Ce nu se stochează pe server
Lista obiectivelor vizitate	Numele utilizatorului	Istoricul locației
Rating-uri individuale pentru fiecare obiectiv	Puncte totale acumulate	Care obiective au fost vizitate și când
<i>Quest-uri</i> completate (care și cum)	Număr total de <i>quest-uri</i> completate	Rating-urile individuale pentru fiecare obiectiv
Rute salvate	Interese selectate	Care <i>quest-uri</i> au fost completate
Identitatea autorului recenziei	-	Autorul fiecărei recenzii sau povești

5 DETALII DE IMPLEMENTARE

Dacă Capitolul 4 descrie ce face fiecare componentă și care au fost motivele din spate pentru fiecare, Capitolul 5 se duce cu un nivel mai jos și detaliază cum s-au implementat concret componentele, prezentându-se, mai întâi, partea de aplicație mobilă, cu serviciul client FL, mecanismul DP, bufferul de interacțiuni și restul componentelor care rulează pe dispozitiv. După aceea, urmează serviciile *backend*, cu agregarea FL, punctele de acces API și modelarea conținutului. Urmează și două secțiuni care sunt în legătură cu partea de ML pentru recomandări, pentru că una descrie fluxul de pre-antrenare pe date publice, prin care modelul global este pregătit înainte de a contribui clienții, iar alta prezintă *framework-ul* de simulare FL pe care se bazează evaluarea din Capitolul 6. De-a lungul capitolului, părți de cod atașate în document arată locurile unde deciziile de implementare sunt cele mai relevante, și anume: antrenamentul local cu limitare a gradientilor, adăugarea zgomotului Gaussian, agregarea FL cu limitare, moderarea printr-un API extern a conținutului și rularea unei runde de simulare. Se adună, în final, dificultățile întâlnite pe parcurs, făcând trecerea spre evaluarea rezultatelor.

5.1 Implementarea aplicației mobile

Implementarea mobilă acoperă șase subsecțiuni, pe care le-am descris în ordinea dependențelor: serviciul client FL, care implementează manual în Dart *forward pass-ul*, partea de *backpropagation* și limitarea gradientilor, fără vreun *framework* de ML. Legat de acest serviciu este mecanismul DP, care adaugă zgomot Gaussian variației de ponderi înainte de trimiterea de către client a actualizărilor, și bufferul local de interacțiuni, care păstrează scorurile de *engagement* între runde și le stochează în *SharedPreferences*. Separat de FL, stocarea offline menține un *cache* al obiectivelor turistice pe telefonul utilizatorilor, la care aplicația poate ajunge când *backend-ul* nu este funcțional sau turistul nu are acces la o conexiune stabilă de rețea. Ultimele două subsecțiuni descriu componente orientate spre utilizator. Integrarea UI a generării rutelor înseamnă, odată, rularea algoritmului pe *backend*, dar și afișarea rutelor, care intră în responsabilitatea aplicației mobile. Sistemul de misiuni cu *geofencing* notifică când turistul intră în raza unui obiectiv și pune interacțiunea în bufferul local.

5.1.1 Serviciul client de învățare federată

Serviciul client este implementat în Dart, în `fl_client_service.dart`, fără alte biblioteci adiționale, deoarece dimensiunea redusă a modelului, de doar 1.281 de parametri (≈ 5 KB), nu necesită adăugarea unui *runtime* de 1,3-30 MB, cum am argumentat în capitolul anterior. Cu scopul de a reutiliza cod, același *forward pass* (metoda `predictScore`) este folosit și pentru a calcula scorul de potrivire afișat sub formă de categorie - scăzut, mediu, ridicat - atunci

când utilizatorul apasă pe un obiectiv pentru a deschide fișa acestuia.

Antrenamentul local cuprinde 5 epoci de SGD pe ce se află în buffer, cu o rată de învățare egală cu 0,01. La fiecare eșantion, după ce se calculează gradientii pentru cele 3 straturi, norma L2 a vectorului acestora este limitată la pragul $\tau = 1$ prin rescalare, fiind vorba de limitarea descrisă la 4.4.4, și, în plus, actualizările cu gradienti NaN sau Inf sunt ignorate, ceea ce previne propagarea valorilor nenumere spre modelul global. În Anexe se regăsește partea de *backpropagation* din metoda `_trainLocally`, în *listing-ul A.1*.

5.1.2 Adăugarea de zgomot pentru asigurarea confidențialității diferențiale

Mecanismul DP este implementat în metoda `_addDPNoise`, metodă care este apelată pe dispozitiv imediat după ce are loc antrenamentul local, înainte să fie trimisă actualizarea spre server. Se calculează, mai întâi, delta - diferența dintre ponderile antrenate și varianta modelului de când runda a început. Apoi, fiecare coordonată a deltei este trunchiată la intervalul $[-\tau, \tau]$ cu $\tau = 1$, pentru a mărgini sensibilitatea. După care, peste fiecare coordonată se adaugă zgomot Gaussian $\mathcal{N}(0, \sigma^2)$ cu $\sigma = \sigma_{\text{noise}} \cdot \tau = 0,1$, ponderile trimise către server fiind reconstruite adăugând delta cu zgomotul aplicat la varianta globală, astfel încât serverul nu primește niciodată delta în stare curată.

Zgomotul Gaussian este generat prin transformata Box-Muller¹, implementarea putându-se observa în metoda `_gaussian`. Transformata produce eșantioane $\mathcal{N}(0, \sigma^2)$ din două variabile uniforme independente $U_1, U_2 \sim \mathcal{U}(0, 1)$, iar valoarea U_1 este trunchiată la 10^{-10} pentru a evita $\log(0)$, dar trunchierile de la acest pas nu trebuie confundate cu limitarea normei L2 de la secțiunea anterioară, deoarece aceea scalează vectorul întreg al gradientilor ca să prevină fenomenul de *exploding gradients*, pe când cea de la pasul de DP ține sensibilitatea mărginită pentru ca zgomotul adăugat să aducă un efect real asupra confidențialității. Implementarea celor două metode, `_addDPNoise` și `_gaussian`, este atașată în *listing-ul A.2* din Anexe.

5.1.3 Bufferul local de interacțiuni și persistența datelor

Interacțiunile dintre runde ale utilizatorului ajung într-un buffer persistent prin Shared-Preferences (`local_data_service.dart`), ele fiind indexate după ID-ul obiectivului turistic sau cultural. Există și un risc, adunarea unor intrări duplicate, așa că, prin structura `_LandmarkBuffer` (`fl_client_service.dart`) se păstrează o singură intrare pentru fiecare obiectiv. Sunt prezente două cazuri pentru acordarea unei etichete de *engagement* (scor) unui obiectiv, primul fiind acordarea unei evaluări sub formă de recenzie (1-5 stele), care suprascrive

¹Box, G. E. P., & Muller, M. E. (1958). A note on the generation of random normal deviates. *The Annals of Mathematical Statistics*, 29(2), 610-611.

orice scor anterior atribuit locației. Pentru celelalte tipuri de interacțiuni, se păstrează cel mai ridicat scor înregistrat pentru obiectiv. De asemenea, persistența prin `SharedPreferences` face ca bufferul să nu fie golit dacă aplicația este oprită înainte ca runda să se încheie - este golit doar după trimiterea cu succes a actualizării spre server. În plus, ecranul de profil afișează numărul curent de interacțiuni din buffer, în acest mod utilizatorul având vizibilitate asupra stării procesului FL local, după cum se poate observa în **figura 9** din Anexe.

5.1.4 Stocarea temporară a obiectivelor pentru funcționarea offline

Lista completă de obiective se salvează tot în `SharedPreferences` (`local_data_service.dart`), sub o cheie globală, care este comună tuturor utilizatorilor, deoarece obiectivele sunt date publice, deci nu este nevoie să individualizăm pentru fiecare utilizator. Acestea nu au un timp de existență predefinit, locațiile fiind suprascrise la fiecare *fetch* reușit. Fluxul legat de interogarea obiectivelor este reprezentat prin metoda `fetchAllLandmarks` (`map_provider.dart`) care solicită prin punctul de acces `/landmarks/all` toate locațiile de la server, iar dacă se întoarce o eroare, se încarcă colecția de obiective din *cache*. Întreaga colecție este stocată unitar, și nu este implementată paginarea bazată pe locația curentă. Aceasta este o limitare care devine ineficientă la scară largă, unde un număr foarte mare de obiective ar face randarea pe hartă imposibilă.

5.1.5 Algoritmul de generare a rutelor

În generarea unei rute este inclus pentru utilizator un panou inferior care expune un parametru configurabil (rata de departajare; 0-1000 metri, implicit 200 de metri), panoul fiind un *slider* între „Shortest path” și „Most relevant”. La apăsarea butonului de generare a rutei, se trimite de la client un *request* de tip POST către `/routes/generate` cu: poziția curentă, numărul maxim de opriri (5), timpul disponibil (300 de minute) și valoarea *slider-ului*. Interesele utilizatorului nu sunt trimise în *request*, fiindcă serverul le preia din profilul utilizatorului din baza de date.

Pe server, obiectivele de pe o rază de 10 km sunt ordonate după un scor inițial $scor_{FL} \times 0.8 + scor_{distance} \times 0.2$, unde scorul FL provine din predicția MLP, iar scorul legat de distanță normalizează distanța față de poziția curentă în intervalul $[0, 1]$. Dacă modelul FL nu este disponibil în Redis pentru a fi folosit la acest pas, scorul de potrivire a intereselor înlocuiește componenta FL cu același coeficient de 80%. Se construiește o rută cu obiectivele selectate în maniera *greedy nearest-neighbour*, cu rata de configurare (`flTiebreakerM`) menționată anterior - oprirea următoare se determină minimizând $dist - flTiebreakerM \times interest_match$, unde *interest_match* se calculează din interesele utilizatorului - ajungându-se la următorul rezultat: un obiectiv mai relevant poate fi ales în locul unuia mai apropiat, dacă se încadrează în raza *slider-ului* setată în panou. Durata atribuită fiecărei opriri pornește dintr-o valoare

aleasă empiric în funcție de tipul obiectivului, și este scalată prin scorul FL al obiectivului în intervalul $[0.75, 1.25]$, astfel că locațiile cu un scor mai mare primesc mai mult timp de vizitare. Modul în care arată o rută generată poate fi vizualizat în **figura 10** din Anexe.

5.1.6 Sistemul de misiuni și tehnologia de *geofencing*

Misiunile se accesează din fișa unui obiectiv, prin butonul „View quests”. La apăsare, aplicația măsoară, pe baza poziției curente, cât de departe se află utilizatorul de obiectiv, iar în cazul în care utilizatorul se află la mai mult de 100 de metri, misiunile apar doar în modul citire, fără posibilitatea de a trimite un răspuns pentru ele. Sub acest prag, misiunea devine activă: utilizatorul răspunde la un *quiz* (întrebare cu variante sau notă text), trimis printr-o cerere HTTP de tip POST la `/quests/{id}/complete`. Pentru un răspuns corect, serverul acordă puncte și misiunea se marchează drept finalizată pe dispozitiv. Când este greșit, obiectivul se marchează, totuși, ca vizitat, de vreme ce utilizatorul este fizic acolo. Panoul de misiuni poate fi observat în **figura 11** din Anexe.

Detectarea apropierii față de locații se face printr-un serviciu de *geofencing* manual (`proximity_service.dart`), construit pe fluxul de poziții GPS cu un filtru de 10 metri, ceea ce înseamnă că fluxul GPS trimite o actualizare doar dacă utilizatorul s-a deplasat cu cel puțin 10 metri față de poziția curentă. Cu fiecare actualizare, se recalculează distanța față de obiectivele din zona curentă - cele din raza de 1500 de metri față de ultima poziție. Evenimentul ENTER se trimite când utilizatorul ajunge la mai puțin de 100 de metri de un obiectiv, iar cel de EXIT când iese din această rază. Dacă utilizatorul s-a deplasat cu mai mult de 200 de metri față de ultimul punct de la care s-a actualizat lista de obiective, aceasta se reîncarcă de la server.

5.2 Implementarea serviciilor *backend*

5.2.1 Serviciul de agregare

5.2.2 Punctele de acces API

5.2.3 Sistemul de recenzii și moderarea conținutului

5.3 Fluxul etapei de pre-antrenare

5.4 *Framework-ul* pentru simularea procesului de învățare federată

5.5 Dificultăți întâmpinate în procesul de implementare

6 EVALUARE

6.1 Corectitudinea funcțională

6.2 Performanța modelului de învățare federată

6.2.1 Rezultatele etapei de pre-antrenare

6.2.2 Rezultatele simulării procesului de învățare federată

6.3 Evaluarea confidențialității, robusteții și scalabilității

6.4 Analiză comparativă și discuții

7 CONCLUZII

7.1 Concluzii generale

7.2 Direcții de cercetare și lucrări viitoare

BIBLIOGRAFIE

- [1] D. A. E. Acar, Y. Zhao, R. M. Navarro, M. Mattina, P. N. Whatmough, and V. Saligrama. Federated learning based on dynamic regularization. In *International Conference on Learning Representations (ICLR)*, 2021. doi:10.48550/arXiv.2111.04263.
- [2] G. Adomavicius and A. Tuzhilin. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*, 17(6):734–749, 2005. doi:10.1109/TKDE.2005.99.
- [3] M. G. Arivazhagan, V. Aggarwal, A. K. Singh, and S. Choudhary. Federated learning with personalization layers. *arXiv preprint arXiv:1912.00818*, 2019. doi:10.48550/arXiv.1912.00818.
- [4] K. Bonawitz, H. Eichner, N. Grieskamp, D. Huba, A. Ingerman, V. Ivanov, C. Kiddon, J. Konecny, S. Mazzocchi, H. B. McMahan, T. Van Overveldt, D. Petrou, D. Ramage, and J. Roselander. Towards federated learning at scale: System design. In *Proceedings of the 2nd SysML Conference*, 2019. doi:10.48550/arXiv.1902.01046.
- [5] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth. Practical secure aggregation for privacy-preserving machine learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017. doi:10.1145/3133956.3133982.
- [6] D. Gavalas, C. Konstantopoulos, K. Mastakas, and G. Pantziou. Mobile recommender systems in tourism. *Journal of Network and Computer Applications*, 39:319–333, 2014. doi:10.1016/j.jnca.2013.04.006.
- [7] P. Kairouz, H. B. McMahan, et al. Advances and open problems in federated learning. *Foundations and Trends in Machine Learning*, 14(1–2), 2021. doi:10.1561/22000000083.
- [8] S. P. Karimireddy, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, and A. T. Suresh. Scaffold: Stochastic controlled averaging for federated learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020. doi:10.48550/arXiv.1910.06378.
- [9] L. Li, Y. Fan, M. Tse, and K.-Y. Lin. A review of applications in federated learning. *Computers & Industrial Engineering*, 149:106854, 2020. doi:10.1016/j.cie.2020.106854.
- [10] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith. Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing Magazine*, 37(3), 2020. doi:10.1109/MSP.2020.2975749.

- [11] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith. Federated optimization in heterogeneous networks. In *Proceedings of Machine Learning and Systems (MLSys)*, 2020. doi:10.48550/arXiv.1812.06127.
- [12] L. Lyu, H. Yu, and Q. Yang. Threats to federated learning: A survey. *arXiv preprint arXiv:2003.02133*, 2020. doi:10.48550/arXiv.2003.02133.
- [13] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017. doi:10.48550/arXiv.1602.05629.
- [14] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang. Learning differentially private recurrent language models. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1710.06963. doi:10.48550/arXiv.1710.06963.
- [15] R. Shokri, M. Stronati, C. Song, and V. Shmatikov. Membership inference attacks against machine learning models. In *IEEE Symposium on Security and Privacy (S&P)*, 2017. doi:10.1109/SP.2017.41.
- [16] K. Wei, J. Li, M. Ding, C. Ma, H. H. Yang, F. Farokhi, S. Jin, T. Q. S. Quek, and H. V. Poor. Federated learning with differential privacy: Algorithms and performance analysis. *IEEE Transactions on Information Forensics and Security*, 15, 2020. doi:10.1109/TIFS.2020.2988575.
- [17] C. Xie, S. Koyejo, and I. Gupta. Asynchronous federated optimization. *arXiv preprint arXiv:1903.03934*, 2019. doi:10.48550/arXiv.1903.03934.
- [18] Q. Yang, Y. Liu, T. Chen, and Y. Tong. Federated machine learning: Concept and applications. *ACM Transactions on Intelligent Systems and Technology*, 10(2):12, 2019. doi:10.1145/3298981.
- [19] C. Zhang, Y. Xie, H. Bai, B. Yu, W. Li, and Y. Gao. A survey on federated learning. *Knowledge-Based Systems*, 216:106775, 2021. doi:10.1016/j.knosys.2021.106775.

ANEXE

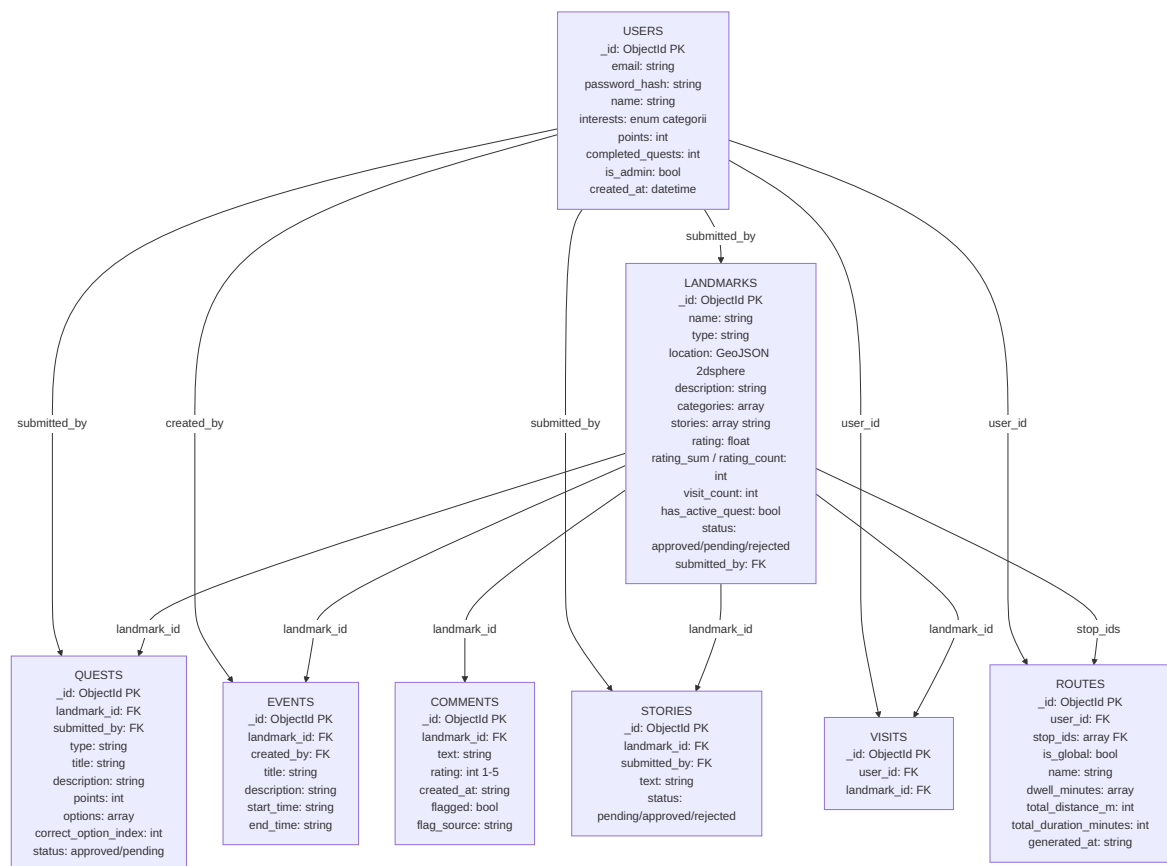


Figura 8: Modelul complet de date al aplicației CultureQuest: colecțiile MongoDB, câmpurile principale și relațiile dintre ele.

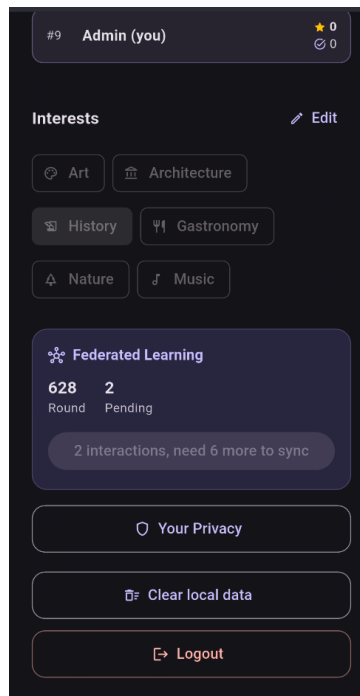


Figura 9: Ecranul de profil cu statusul FL și numărul de interacțiuni din buffer.

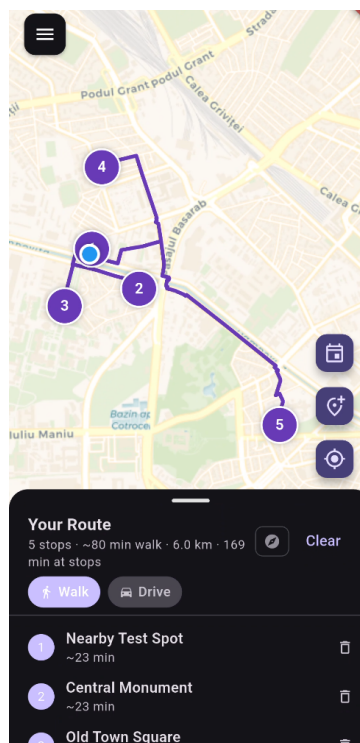


Figura 10: Ruta generată pe hartă.

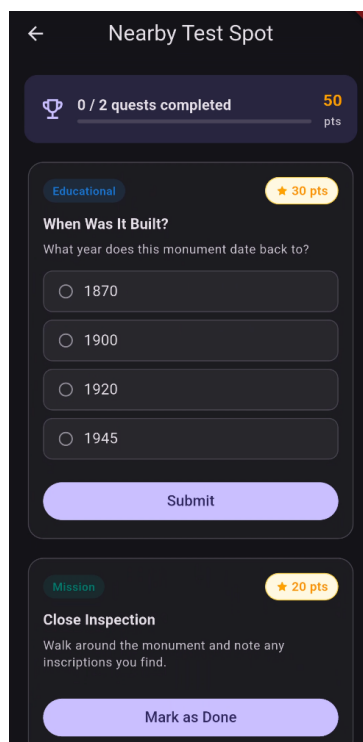


Figura 11: Panoul de misiuni (*quest-uri*) disponibil la detectarea unui obiectiv în apropiere.

A EXTRASE DE COD

Listing A.1: Backpropagare și limitare a gradientilor în `_trainLocally` (`fl_client_service.dart:272-302`).

```
// Backward
final dLoss = 2.0 * (out - y);
final dSig = out * (1.0 - out);
final dz3 = [dLoss * dSig];

final dW3 = _outerVec(dz3, a2);
final db3 = List<double>.from(dz3);
final da2 = _vecMatT(dz3, w3);
final dz2 = List.generate(16, (i) => da2[i] * _reluDeriv(z2[i]));

final dW2 = _outerVec(dz2, a1);
final db2g = List<double>.from(dz2);
final da1 = _vecMatT(dz2, w2);
final dz1 = List.generate(32, (i) => da1[i] * _reluDeriv(z1[i]));

final dW1 = _outerVec(dz1, x);
final db1g = List<double>.from(dz1);

if (_hasNaN(db1g) || _hasNaN(db2g) || _hasNaN(db3)) continue;

final allGrads = [
  ...dW1.expand((r) => r), ...db1g,
  ...dW2.expand((r) => r), ...db2g,
  ...dW3.expand((r) => r), ...db3,
];
final norm = math.sqrt(allGrads.fold(0.0, (s, v) => s + v * v));
final scale = (norm > 1.0) ? 1.0 / norm : 1.0;

_applyGrad(w1, dW1, lr * scale);
_applyGradVec(b1, db1g, lr * scale);
_applyGrad(w2, dW2, lr * scale);
_applyGradVec(b2, db2g, lr * scale);
_applyGrad(w3, dW3, lr * scale);
_applyGradVec(b3, db3, lr * scale);
```

Listing A.2: Adăugarea zgomotului Gaussian pentru fiecare coordonată a deltei (fl_client_service.dart:348-373).

```
void _addDPNoise(List<List<double>> globalSnapshot) {
    final noiseStd = kDPSigma * kDPClipNorm;
    for (int l = 0; l < _weights.length; l++) {
        final delta = List<double>.generate(
            _weights[l].length,
            (i) => (_weights[l][i] - globalSnapshot[l][i])
                .clamp(-kDPClipNorm, kDPClipNorm),
        );
        for (int i = 0; i < delta.length; i++) {
            delta[i] += _gaussian(noiseStd);
        }
        for (int i = 0; i < _weights[l].length; i++) {
            _weights[l][i] = globalSnapshot[l][i] + delta[i];
        }
    }
}

double _gaussian(double sigma) {
    final u1 = _rng.nextDouble().clamp(1e-10, 1.0);
    final u2 = _rng.nextDouble();
    final z = math.sqrt(-2.0 * math.log(u1))
        * math.cos(2.0 * math.pi * u2);
    return z * sigma;
}
```